

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ОТЧЕТ

о преддипломной (производственной) практике

наименование вида и типа практики

на (в) ООО «Предприятие ВТИ-Сервис»

наименование предприятия, организации, учреждения

Студента 4 курса, группы ПО-226

курса, группы

Алешина Кирилла Романовича

фамилия, имя, отчество

Руководитель практики от
предприятия, организации,
учреждения

Оценка _____

должность, звание, степень

фамилия и. о.

подпись, дата

Руководитель практики от
университета

Оценка _____

К.Т.Н. доцент

должность, звание, степень

Чаплыгин А. А.

фамилия и. о.

подпись, дата

Члены комиссии

подпись, дата

фамилия и. о.

подпись, дата

фамилия и. о.

подпись, дата

фамилия и. о.

Курск 2026 г.

СОДЕРЖАНИЕ

1	Анализ предметной области	5
1.1	Интерактивные приложения как класс программных систем	5
1.2	Программные средства построения интерактивных приложений	7
1.3	Компонентно-ориентированный подход к построению интерактивных приложений	9
1.4	Ключевые сущности предметной области	11
1.5	Особенности разработки интерактивных приложений на Java	13
1.6	Анализ существующих программных решений	14
1.7	Выводы по анализу предметной области	17
2	Техническое задание	19
2.1	Основание для разработки	19
2.2	Цель и назначение разработки	19
2.3	Требования к программной системе	21
2.3.1	Требования к архитектурной модели	21
2.3.2	Требования к управлению приложением и сценами	22
2.3.3	Требования к графической подсистеме	23
2.3.4	Требования к пользовательскому вводу	23
2.3.5	Требования к системе пользовательских скриптов	24
2.3.6	Требования к базовой физической подсистеме	24
2.3.7	Требования к демонстрационной сцене	25
2.4	Руководство по использованию программной системы	25
2.4.1	Подготовка окружения	26
2.4.2	Создание приложения	27
2.4.3	Создание сцены и добавление систем	28
2.4.4	Добавление источника света	29
2.4.5	Создание камеры	29
2.4.6	Создание статического объекта	30
2.4.7	Создание динамического физического тела	30
2.4.8	Создание кинематического объекта	31

2.4.9	Создание пользовательского скрипта	32
2.4.10	Использование пользовательского ввода	33
2.4.11	Создание отладочной геометрии	33
2.4.12	Общий порядок создания сцены	34
2.5	Требования к программному интерфейсу	35
2.6	Нефункциональные требования	36
2.6.1	Требования к программному обеспечению	36
2.6.2	Требования к аппаратному обеспечению	36
2.6.3	Требования к надежности	36
2.6.4	Требования к сопровождаемости	37
2.6.5	Требования к безопасности	37
2.7	Требования к оформлению документации	38
3	Технический проект	39
3.1	Общие сведения о программной системе	39
3.2	Обоснование выбора технологий	41
3.2.1	Язык программирования Java	42
3.2.2	Библиотека LWJGL	42
3.2.3	OpenGL и GLSL	43
3.2.4	GLFW и STB	43
3.3	Внутреннее устройство программной системы	44
3.3.1	Структура пакетов проекта	44
3.3.2	Основные элементы внутреннего устройства	45
3.3.3	Устройство приложения и контекста выполнения	47
3.3.4	Устройство сцены	48
3.3.5	Сущности, компоненты и системы обработки	49
3.3.6	Жизненный цикл приложения и сцены	50
3.4	Реализация основных подсистем	52
3.4.1	Подсистема управления сценами	52
3.4.2	Графическая подсистема	52
3.4.3	Подсистема ресурсов	55
3.4.4	Подсистема пользовательского ввода	55

3.4.5	Подсистема пользовательских скриптов	55
3.4.6	Подсистема отладочной визуализации	56
3.5	Реализация физической подсистемы	56
3.5.1	Общая структура физической подсистемы	56
3.5.2	Математическая модель движения	58
3.5.3	Метод численного интегрирования	58
3.5.4	Обнаружение столкновений	59
3.5.5	Разрешение столкновений	60
3.5.6	Границы текущего этапа разработки	62
3.6	Демонстрационная сцена PhysicsScene	62
3.7	Выводы по разделу	65
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	65

1 Анализ предметной области

1.1 Интерактивные приложения как класс программных систем

Интерактивным приложением называют программную систему, результат работы которой зависит не только от заранее заданных данных, но и от постоянного взаимодействия с пользователем. Такое приложение получает события от клавиатуры, мыши или другого устройства ввода, изменяет внутреннее состояние и выводит обновленное изображение с достаточно малой задержкой, чтобы пользователь воспринимал происходящее как непрерывный процесс.

К интерактивным приложениям относятся компьютерные игры, учебные симуляторы, демонстрационные 3D-сцены, инженерные визуализаторы, виртуальные лаборатории, средства прототипирования интерфейсов и интерактивные художественные инсталляции. Эти программы различаются по назначению, но имеют общую техническую основу. Для них характерны периодическое обновление состояния, отрисовка кадра, обработка ввода и управление набором объектов, присутствующих в сцене.

Обычное прикладное приложение часто строится вокруг запроса пользователя: пользователь нажимает кнопку, программа выполняет действие и возвращается в состояние ожидания. Интерактивное приложение работает иначе. В нем присутствует непрерывный цикл: программа опрашивает устройства ввода, рассчитывает изменения за прошедший промежуток времени, обновляет положение и состояние объектов, после чего формирует новый кадр. Такой цикл принято называть главным циклом приложения или игровым циклом, даже если разрабатываемая программа не является игрой.

В простейшем виде работа интерактивного приложения может быть представлена схемой, приведенной на рисунке 1.1.



Рисунок 1.1 – Обобщенная схема работы интерактивного приложения

На практике каждый из показанных этапов распадается на несколько подзадач. Обработка ввода включает регистрацию нажатий клавиш, перемещения мыши, изменения состояния кнопок и колесика прокрутки. Обновление состояния объектов связано с вычислением трансформаций, анимаций, перемещений, пользовательских сценариев поведения. Обработка столкновений требует определения пересечений между объектами, реакции на контакт и, при необходимости, изменения скорости или положения объектов. Отрисовка кадра предполагает подготовку данных для графического процессора, выбор камеры, материалов, источников света, шейдеров и очередности вывода объектов.

Особенность таких приложений состоит в том, что их внутренняя структура быстро усложняется с ростом числа объектов. Один объект может иметь положение в пространстве, графическое представление, коллайдер, физическое тело, пользовательский сценарий поведения, источник света или камеру. При этом разные объекты используют разные сочетания этих возможностей. Например, камера не должна иметь графическую модель, а декоративный объект может не участвовать в столкновениях. Если каждую разновидность объекта описывать отдельным классом, то количество классов и связей между ними начинает расти быстрее, чем сама предметная об-

ласть. Именно поэтому при разработке интерактивных приложений особенно важен выбор архитектурного подхода.

1.2 Программные средства построения интерактивных приложений

Разработка интерактивного приложения с нуля требует решения большого числа технических задач, не связанных напрямую с логикой конкретного проекта. Разработчику нужно создать окно, получить графический контекст, организовать цикл обновления, загрузить ресурсы, отрисовать геометрию, обработать ввод, хранить состояние сцены, описать поведение объектов и обеспечить освобождение ресурсов. Поэтому на практике редко используют только стандартную библиотеку языка программирования. Обычно выбирают один из классов программных средств: низкоуровневую графическую библиотеку, фреймворк, игровой движок или специализированную среду прототипирования.

Низкоуровневая графическая библиотека предоставляет доступ к аппаратно-ускоренной графике, но почти не навязывает архитектуру приложения. Примером является OpenGL. Khronos Group описывает OpenGL как кроссплатформенный API для отрисовки 2D- и 3D-графики [17]. Такой API решает задачу взаимодействия с графическим процессором, но не предоставляет готовой модели сцены, системы сущностей, иерархии объектов или компонентной архитектуры. Программист получает контроль над рендерингом, но вынужден самостоятельно проектировать остальную инфраструктуру приложения.

Для языка Java важную роль играет LWJGL. Официальный сайт характеризует LWJGL как библиотеку, предоставляющую Java-приложениям кроссплатформенный доступ к нативным API, включая OpenGL, OpenAL и OpenCL [11]. Фактически LWJGL является мостом между Java и низкоуровневыми средствами работы с графикой, звуком и окном. Это делает библиотеку удобной основой для собственных движков и графических инструментов, но она не является готовой системой построения сцен. Разработчику необхо-

димо самостоятельно реализовывать структуру объектов, управление ресурсами, обновление состояния и пользовательские абстракции.

Фреймворк находится на более высоком уровне. Он не только предоставляет доступ к графике и вводу, но и задает часть структуры приложения. Одним из наиболее известных Java-фреймворков является libGDX. В официальном описании libGDX определяется как кроссплатформенный Java-фреймворк для разработки игр на основе OpenGL ES, поддерживающий Windows, Linux, macOS, Android, браузер и iOS [23]. Его сильная сторона заключается в переносимости и широком наборе готовых средств. Однако libGDX оставляет разработчику свободу в выборе архитектуры предметных объектов. Для небольших проектов это удобно, но при построении системы из множества объектов разработчик все равно должен заранее решить, как именно будут описываться сущности, компоненты и связи между ними.

Игровой движок поднимает уровень абстракции еще выше. Он обычно содержит модель сцены, подсистемы рендеринга, ввода, ресурсов, анимации, физики и инструменты для разработки. Для Java таким решением является jMonkeyEngine. На официальном сайте он описан как современный игровой движок, написанный преимущественно на Java и ориентированный на разработчиков, которым нужен контроль над кодом и возможность расширять движок под собственный процесс разработки [24]. Это полноценный инструмент, но вместе с полнотой возможностей появляется и более высокий порог входа. Пользователь должен принять архитектурные решения движка, изучить его жизненный цикл, систему ресурсов, пространственную модель и набор классов.

Отдельную группу составляют среды прототипирования и визуального программирования. Processing, согласно официальному описанию, является гибким программным «скетчбуком» и языком для обучения программированию, который с 2001 года используется в визуальных искусствах, образовании и быстром прототипировании [25]. Такие инструменты снижают барьер входа, но их модель обычно ориентирована на небольшие эскизы, визуальные эксперименты и учебные проекты. Для разработки расширяемой ком-

понентной системы они подходят хуже, чем библиотека или фреймворк, поскольку скрывают часть инфраструктуры и задают собственный стиль организации программы.

Таким образом, между низкоуровневой библиотекой и полноценным игровым движком существует промежуточная ниша. Java-разработчику может быть нужна не готовая тяжелая среда с редактором и большим набором подсистем, а легковесная программная основа, позволяющая строить интерактивные сцены из сущностей, компонентов и систем. Такая основа должна закрывать типовые задачи цикла приложения, сцены, рендеринга, ввода и базовой обработки столкновений, но при этом оставаться понятной и расширяемой на уровне исходного кода.

1.3 Компонентно-ориентированный подход к построению интерактивных приложений

Одним из главных вопросов при проектировании интерактивного приложения является способ описания объектов сцены. В традиционном объектно-ориентированном подходе разработчик часто начинает с иерархии классов: базовый класс объекта, затем классы персонажей, снарядов, декораций, источников света, камер, физических объектов. На раннем этапе такая схема кажется удобной. Например, можно создать базовый класс `GameObject`, от него унаследовать `RenderableObject`, затем `PhysicalRenderableObject` и так далее.

Проблема появляется, когда свойства начинают комбинироваться. Объект может быть видимым, но не иметь физического тела. Другой объект может участвовать в столкновениях, но не отображаться на экране. Третий объект может иметь скрипт поведения и источник света. При наследовании такие сочетания приводят либо к очень глубокой иерархии, либо к дублированию кода в разных ветвях. Изменение одного свойства становится изменением класса, а не добавлением новой возможности к объекту.

Компонентный подход решает эту проблему иначе. Объект сцены рассматривается не как экземпляр специализированного класса, а как контейнер для набора независимых компонентов. Компонент описывает одну характе-

ристику или одну сторону поведения объекта: положение в пространстве, камеру, отрисовку сетки, источник света, физическое тело, коллайдер, пользовательский скрипт. Чтобы изменить объект, разработчик добавляет или удаляет компоненты, не создавая новый класс для каждого сочетания возможностей.

Преимущество компонентной модели хорошо видно на простом примере. Объект, имеющий компоненты Transform и MeshRenderer, может быть отрисован как обычная модель. Если к нему добавить компонент Rigidbody, он начинает участвовать в физической обработке. Если добавить компонент Script, объект получает пользовательское поведение. При этом базовая сущность остается той же самой, а набор возможностей определяется составом компонентов.

В популярных инструментах разработки интерактивных приложений компонентная идея встречается достаточно часто. Например, в Unity функциональность объекта сцены определяется компонентами, прикрепленными к GameObject [26]. Unity также развивает отдельное направление ECS, называя его дата-ориентированным фреймворком для высокопроизводительной обработки сущностей [27]. Однако Unity не является Java-решением и представляет собой крупную экосистему с собственным редактором, форматом проектов и моделью разработки. Для Java-разработчика, желающего изучить или использовать компонентную модель в небольшом приложении, такой инструмент может оказаться избыточным.

Компонентно-ориентированную архитектуру необходимо отличать от канонического Entity-Component-System. В строгом варианте ECS сущность обычно является только идентификатором, компонент хранит данные, а вся логика выполняется системами, которые выбирают сущности по набору компонентов. Такой подход хорошо подходит для высокопроизводительных сцен, поскольку данные можно располагать в памяти компактно и обрабатывать пакетами. Но он требует более строгой организации данных и усложняет разработку для небольших проектов.

Разрабатываемая программная система ближе к компонентно-ориентированной модели с элементами ECS, чем к полностью каноническому дата-ориентированному ECS. В ней есть сущности, компоненты, системы и запросы к сущностям по составу компонентов. При этом компоненты являются объектами Java и могут иметь собственный жизненный цикл. Такой вариант сохраняет основную идею композиции объектов из компонентов, но остается более привычным для Java-разработчика, работающего с объектами, классами и методами.

Для ВКР выбран именно этот подход, так как он позволяет совместить простоту объектной модели Java с гибкостью компонентной структуры сцены. Разработчик получает возможность описывать интерактивные объекты через состав компонентов, не создавая сложную иерархию наследования и не переходя к полностью дата-ориентированной архитектуре.

1.4 Ключевые сущности предметной области

Несмотря на различие конкретных библиотек и движков, большинство средств построения интерактивных приложений оперирует схожими понятиями. Ключевой сущностью является сцена. Она задает контекст выполнения: содержит набор объектов, управляет их жизненным циклом, обновляет системы и определяет, какие объекты участвуют в текущем кадре. В играх сцена может соответствовать уровню, меню или отдельному режиму. В учебном симуляторе сцена может представлять лабораторную установку, эксперимент или демонстрационный пример.

Сущность представляет отдельный объект сцены. Сама по себе сущность может не содержать логики и данных предметной области. Ее назначение состоит в том, чтобы объединять компоненты и выступать единицей идентификации. Такой подход позволяет рассматривать любой объект сцены одинаково: камеру, куб, источник света, физическое тело или служебный объект управления.

Компонент описывает отдельную характеристику сущности. Наиболее универсальным компонентом является трансформация, задающая положе-

ние, поворот и масштаб объекта. Без нее невозможно определить, где находится модель, камера или коллайдер. Другие компоненты добавляют графическое представление, камеру, освещение, физическое тело, обработчик столкновений или пользовательское поведение.

Система выполняет обработку сущностей, имеющих определенный набор компонентов. Например, система рендеринга выбирает сущности с трансформацией и графическим компонентом, система скриптов вызывает пользовательское поведение, а физическая система работает с физическими телами и коллайдерами. Благодаря этому логика обработки не размазывается по всем объектам, а сосредотачивается в подсистемах, отвечающих за конкретную область.

Ресурсы приложения включают шейдеры, текстуры, материалы, меши и другие данные, используемые несколькими объектами. В интерактивных приложениях важно не только загрузить ресурс, но и контролировать срок его жизни. Графические ресурсы занимают память не только в оперативной памяти приложения, но и на стороне графического процессора, поэтому при смене сцены или завершении приложения их необходимо корректно освободить.

Камера определяет способ наблюдения сцены. В 3D-приложении она хранит параметры проекции, область видимости, соотношение сторон и положение в пространстве. Источники света влияют на расчет освещенности объектов. Материал и шейдер задают способ визуализации поверхности. Ввод пользователя связывает внешние действия с изменениями состояния сцены.

Связь этих сущностей можно описать так: приложение содержит активную сцену, сцена содержит сущности и системы, сущности состоят из компонентов, а системы обрабатывают подходящие сущности и используют ресурсы для формирования результата. Такая модель достаточно проста для понимания, но при этом позволяет строить разнообразные интерактивные приложения без создания отдельного класса для каждого вида объекта.

1.5 Особенности разработки интерактивных приложений на Java

Java традиционно используется для разработки серверных приложений, корпоративных систем, инструментов автоматизации, учебных программ и кроссплатформенных настольных приложений. Для интерактивной графики Java применяется реже, чем C++ или C#, однако у нее есть ряд свойств, полезных для рассматриваемой задачи. Язык имеет строгую типизацию, развитую стандартную библиотеку, автоматическое управление памятью и широкий набор средств разработки. Официальная документация Java SE 25 включает спецификации платформы, API и инструменты JDK, что делает платформу достаточно полно документированной для учебного и прикладного использования [8].

Главное преимущество Java в контексте данной работы заключается в понятной объектной модели. Сущности, компоненты, системы, материалы, меши, камеры и ресурсы естественно выражаются через классы и интерфейсы. Это упрощает проектирование программной системы, ориентированной на разработчика, который уже знаком с Java и хочет создавать интерактивные сцены без перехода на другой язык и без изучения крупной игровой среды.

При этом Java не предоставляет встроенного доступа к OpenGL на уровне стандартной библиотеки. Для работы с аппаратно-ускоренной графикой нужны дополнительные библиотеки, такие как LWJGL. Использование LWJGL позволяет сохранить разработку на Java, но работать с графическим API, близким к нативному уровню. Такой подход требует большей аккуратности: разработчик должен учитывать создание окна, контекста OpenGL, буферов, шейдеров, загрузку данных на графический процессор и освобождение ресурсов.

Еще одна особенность связана с производительностью. Интерактивное приложение должно обновлять состояние и отрисовывать кадры много раз в секунду. Поэтому неудачные архитектурные решения быстро становятся заметными: лишние выделения памяти, частый поиск объектов, неоптимальная сортировка команд рендеринга или избыточная работа с графическими ре-

сурсами приводят к задержкам. В Java часть низкоуровневых деталей скрыта виртуальной машиной, но это не отменяет необходимости аккуратно строить цикл обновления и ограничивать лишние операции внутри него.

Для рассматриваемой программной системы важно, что математика трехмерной сцены реализуется в самом проекте. Векторные, матричные и кватернионные операции используются для трансформаций, камеры, ориентации объектов и расчета положения в пространстве. Такой подход увеличивает объем реализации, но делает систему более самостоятельной и позволяет подробно раскрыть проектные решения в техническом проекте и рабочем проекте.

Отсутствие готового сборщика на раннем этапе разработки не влияет на анализ предметной области, но отражает текущий уровень зрелости программной системы. Для дальнейшего развития проекта целесообразно добавить Maven или Gradle, чтобы упростить подключение зависимостей, запуск тестов и воспроизводимую сборку. В рамках первой главы этот вопрос важен как подтверждение общей проблемы: Java-разработчику нужны не только низкоуровневые библиотеки, но и удобная, воспроизводимая инфраструктура создания интерактивных приложений.

1.6 Анализ существующих программных решений

Для сравнения выбраны решения, близкие к теме работы по разным признакам. LWJGL представляет низкоуровневый доступ к графическим и системным API из Java. libGDX является Java-фреймворком для кроссплатформенной разработки. jMonkeyEngine представляет полноценный Java-движок. Processing отражает подход быстрого визуального прототипирования на Java-подобной основе. Unity включен в анализ не как Java-решение, а как распространенный пример компонентной модели и практики построения интерактивных сцен.

Краткое сравнение программных средств построения интерактивных приложений приведено в таблице 1.1.

Таблица 1.1 – Сравнение программных средств построения интерактивных приложений

Средство	Тип и платформа	Возможности	Особенности для Java-разработчика
LWJGL	Низкоуровневая Java-библиотека, использующая нативные привязки	Доступ к OpenGL, OpenAL и другим системным API; построение 2D- и 3D-графики возможно через OpenGL; собственного редактора нет	Дает высокий уровень контроля над графикой и окружением выполнения, но не предоставляет готовую сцену, компонентную модель, управление ресурсами и жизненный цикл объектов.
libGDX	Java-фреймворк для разработки игр и графических приложений	Поддерживает 2D- и 3D-графику, работу с ресурсами, вводом и несколькими целевыми платформами; обязательного редактора нет	Удобен для кроссплатформенной разработки, однако архитектура игровых объектов и способ организации логики во многом остаются ответственностью разработчика.
jMonkeyEngine	Java-движок для создания трехмерных приложений	Ориентирован преимущественно на 3D-графику, содержит готовые подсистемы сцены, материалов, освещения, физики и ресурсов; имеются дополнительные инструменты разработки	Близок к полноценному решению на Java, но требует изучения крупного фреймворка и принятия его проектной модели, что может быть избыточно для небольших учебных и исследовательских проектов.

Продолжение таблицы 1.1

Средство	Тип и платформа	Возможности	Особенности для Java-разработчика
Processing	Среда и библиотека для визуального программирования на Java-подобной основе	Позволяет быстро создавать 2D- и 3D-скетчи, визуализации и учебные интерактивные примеры; имеет собственную среду разработки	Имеет низкий порог входа, но больше ориентирован на визуальные эксперименты и прототипирование, чем на построение расширяемой компонентной архитектуры.
Unity	Крупный игровой движок и редактор, использующий язык C#	Поддерживает 2D- и 3D-графику, физику, визуальный редактор сцен, компонентную модель объектов и развитую экосистему инструментов	Показывает преимущества компонентного подхода, но не относится к Java-экосистеме и может быть слишком тяжелым решением для небольших проектов, где требуется простая программная основа.

Из сравнения видно, что существующие решения закрывают разные части задачи. LWJGL удобен как низкоуровневая основа, но не решает архитектурные вопросы. libGDX снижает объем технической работы и дает готовый фреймворк, но не ставит компонентную модель в центр архитектуры. jMonkeyEngine наиболее близок к полноценному движку на Java, однако его использование предполагает освоение более крупной системы. Processing удобен для быстрых визуальных экспериментов, но не ориентирован на построение расширяемой сценной архитектуры. Unity показывает преимуще-

ства компонентного подхода, но относится к другой технологической экосистеме.

Для небольшого Java-проекта возникает разрыв между двумя крайностями. С одной стороны, можно взять низкоуровневую библиотеку и написать всю архитектуру самостоятельно. С другой стороны, можно использовать полноценный движок, приняв его сложность и набор инструментов. В рамках ВКР интерес представляет промежуточное решение: программная система, которая предоставляет компонентную организацию сцены, базовые системы рендеринга, ввода, скриптов и обработки столкновений, но остается достаточно легкой для изучения и модификации.

1.7 Выводы по анализу предметной области

Интерактивные приложения отличаются от обычных прикладных программ наличием постоянного цикла обработки ввода, обновления состояния и вывода кадра. Для их разработки требуется не только реализовать предметную логику, но и создать инфраструктуру сцены, ресурсов, объектов, компонентов, рендеринга и пользовательского ввода.

Анализ существующих решений показал, что Java-разработчик может использовать низкоуровневые библиотеки, фреймворки или полноценные движки. Однако низкоуровневые библиотеки требуют самостоятельной разработки архитектуры, а крупные движки часто оказываются избыточными для небольших учебных, демонстрационных и исследовательских проектов. Это создает потребность в промежуточной программной системе, которая закрывает основные инфраструктурные задачи, но остается простой для изучения и расширения.

Компонентно-ориентированный подход позволяет отказаться от сложных иерархий наследования и описывать объекты сцены как набор независимых компонентов. Для выбранной темы это особенно важно, поскольку объекты интерактивной сцены отличаются не типом класса, а набором возможностей: одни отображаются, другие участвуют в столкновениях, третьи выполняют пользовательскую логику или задают камеру.

Разрабатываемая программная система должна быть ориентирована на Java-разработчика и предоставлять базовый набор средств для построения интерактивных сцен: управление сценой, сущностями, компонентами и системами, рендеринг, обработку ввода, скрипты поведения и базовую обработку столкновений. Такой результат соответствует утвержденной теме ВКР и создает основу для дальнейшего развития системы до более полноценного игрового или интерактивного движка.

2 Техническое задание

2.1 Основание для разработки

Полное наименование системы: «Разработка компонентно-ориентированной программной системы для построения интерактивных приложений на Java».

Основанием для разработки программы является задание на выпускную квалификационную работу бакалавра по направлению подготовки 09.03.04 «Программная инженерия». Разрабатываемая программная система представляет собой программную основу для создания интерактивных приложений на языке Java. В дальнейшем по мере расширения подсистем, средств разработки и пользовательской документации данная система может быть развита до более полного игрового движка, однако в рамках настоящей работы основной акцент сделан на ядре, компонентной модели, сценах, системах обработки, визуализации, вводе и базовой физике.

Разработка ведется с учетом требований к стадиям создания программной документации. В разделе технического задания необходимо определить назначение разработки, сформулировать задачи, описать требования к программной системе и показать порядок ее использования разработчиком интерактивного приложения.

2.2 Цель и назначение разработки

Целью разработки является создание компонентно-ориентированной программной системы, позволяющей Java-разработчику строить простые интерактивные приложения и учебные игровые прототипы без самостоятельной реализации полного набора низкоуровневых механизмов: создания окна, основного цикла приложения, управления сценой, хранения объектов, обработки пользовательского ввода, визуализации трехмерных объектов и базовой обработки физических тел.

Функциональное назначение системы состоит в предоставлении разработчику набора классов и подсистем, с помощью которых можно описывать

интерактивную сцену как совокупность сущностей, компонентов и систем обработки. Сущность используется как контейнер компонентов, компонент хранит состояние или поведение объекта, а система выполняет обработку групп сущностей, удовлетворяющих заданным условиям. Такой подход позволяет отделить общие механизмы работы приложения от прикладной логики конкретной сцены.

Эксплуатационное назначение системы заключается в использовании ее Java-разработчиком при создании настольных интерактивных приложений. Пользователь разрабатываемой системы не является конечным пользователем готовой игры или демонстрации. В данном случае пользователем выступает программист, который подключает исходные коды движка, создает класс приложения, описывает сцену, добавляет объекты, настраивает компоненты и запускает приложение в среде разработки или из командной строки.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Разработать ядро компонентно-ориентированной модели, включающее сущности, компоненты, системы обработки, запросы сущностей и жизненный цикл объектов сцены.
2. Спроектировать и реализовать подсистему управления приложением и сценами, обеспечивающую создание окна, выполнение основного цикла, обработку времени кадра и безопасное изменение состава сцены во время работы программы.
3. Реализовать графическую подсистему на основе OpenGL, обеспечивающую отображение объектов сцены с использованием мешей, материалов, текстур, камер и направленного освещения.
4. Реализовать подсистемы пользовательского ввода, скриптов и базовой физики, позволяющие обрабатывать действия пользователя, задавать поведение объектов средствами Java-кода и демонстрировать взаимодействие физических тел и коллайдеров.
5. Подготовить демонстрационную сцену и описание порядка использования программной системы с примерами кода, показывающими создание

приложения, сцены, сущностей, компонентов, систем обработки и пользовательских скриптов.

2.3 Требования к программной системе

Разрабатываемая программная система должна предоставлять Java-разработчику средства для создания интерактивного приложения программным способом. В системе не предусматривается визуальный редактор сцен, поэтому создание объектов, настройка компонентов, подключение систем и запуск сцены выполняются в исходном коде приложения.

Основным сценарием использования системы является разработка интерактивной сцены. Разработчик создает объект приложения, задает параметры окружения, формирует сцену, добавляет в нее системы обработки, затем создает сущности с компонентами и запускает основной цикл выполнения. После запуска система самостоятельно выполняет обновление активной сцены, обработку ввода, вызов пользовательских скриптов, физическое обновление, рендеринг и освобождение ресурсов при завершении работы.

2.3.1 Требования к архитектурной модели

Архитектурная модель системы должна основываться на компонентно-ориентированном подходе. В системе должны использоваться следующие основные понятия:

- сущность – объект сцены, имеющий уникальный идентификатор и набор компонентов;
- компонент – часть состояния или поведения сущности;
- система – объект, выполняющий обработку сущностей, имеющих требуемый набор компонентов;
- сцена – контейнер сущностей и систем, обновляемый в основном цикле приложения;
- контекст движка – объект, объединяющий окно, ввод, менеджер ресурсов, менеджер сцен, конфигурацию и время выполнения.

Компонентная модель должна позволять добавлять к сущности несколько независимых возможностей. Например, объект сцены может одновременно иметь компонент трансформации, компонент отображения, физическое тело и коллайдер. При этом прикладная логика не должна требовать создания отдельного класса-наследника для каждого нового типа игрового объекта.

Система должна поддерживать жизненный цикл компонентов. После присоединения компонента к сущности должен вызываться этап инициализации связи с сущностью, при запуске сцены – этап начала работы, при остановке – этап остановки, при удалении – этап уничтожения. Это необходимо для корректной регистрации внутренних состояний, освобождения ресурсов и прекращения работы скриптов.

2.3.2 Требования к управлению приложением и сценами

Программная система должна обеспечивать создание окна приложения, настройку базовых параметров запуска и выполнение основного цикла. К базовым параметрам относятся цвет очистки окна, масштаб времени, фиксированный шаг обновления, автоматическое обновление соотношения сторон камеры и режим отладочной проверки ошибок OpenGL.

Менеджер сцен должен обеспечивать загрузку активной сцены и замену текущей сцены во время выполнения. Смена сцены должна выполняться безопасно, без нарушения текущего цикла обновления. Это важно для интерактивных приложений, в которых сцена может быть перезапущена по действию пользователя, например при нажатии клавиши сброса демонстрации.

Сцена должна поддерживать добавление и удаление сущностей, добавление и удаление систем обработки, выполнение цикла обновления и освобождение ресурсов. Изменение структуры сцены во время обновления должно откладываться до безопасного момента, чтобы добавление или удаление объектов из пользовательского скрипта не приводило к ошибкам обхода коллекций.

2.3.3 Требования к графической подсистеме

Графическая подсистема должна использовать OpenGL и обеспечивать отображение трехмерных объектов в окне приложения. Для типового объекта сцены необходимо поддерживать компонент трансформации, компонент отображения меша и материал. Меш должен задавать геометрию объекта, материал – параметры отображения, а трансформация – положение, поворот и масштаб в сцене.

В составе системы должны быть предусмотрены стандартные геометрические примитивы, необходимые для демонстрационных сцен. К таким примитивам относятся куб, плоскость или четырехугольник, а также простые формы, позволяющие быстро собрать тестовую сцену без загрузки модели из внешнего файла. Загрузка мешей из внешних файлов в рамках текущей версии системы не является обязательной возможностью.

Подсистема ресурсов должна обеспечивать загрузку и повторное использование шейдеров и текстур. Текстуры должны загружаться из файлов ресурсов с использованием средств LWJGL STB. Повторная загрузка уже использованного ресурса должна избегаться за счет кэширования, так как в интерактивных приложениях один и тот же шейдер или одна и та же текстура могут использоваться многими объектами.

Для освещения должна быть предусмотрена поддержка базового направленного источника света. Такая модель освещения достаточна для демонстрации объемных объектов, положения камеры и работы материалов, но не рассматривается как полноценная система освещения с разными типами источников, тенями и постобработкой.

2.3.4 Требования к пользовательскому вводу

Программная система должна обеспечивать получение состояния клавиатуры и мыши. Разработчик должен иметь возможность проверять факт удержания клавиши, однократное нажатие клавиши, положение курсора и изменение положения курсора между кадрами. Эти данные должны быть до-

ступны из пользовательских скриптов, поскольку именно скрипты отвечают за прикладное поведение объектов сцены.

Для демонстрационной сцены должно быть предусмотрено управление камерой. Перемещение камеры должно выполняться с помощью клавиш W, A, S, D, SPACE и LEFT_SHIFT; поворот камеры должен выполняться по смещению мыши. Такой способ управления является привычным для интерактивных трехмерных приложений и позволяет свободно осматривать сцену.

2.3.5 Требования к системе пользовательских скриптов

Система должна предоставлять базовый класс сценария поведения. Пользовательский скрипт должен подключаться к сущности как обычный компонент. При запуске сцены система скриптов должна находить такие компоненты и вызывать методы их жизненного цикла.

В пользовательском скрипте разработчик должен иметь возможность получать компоненты текущей сущности, читать пользовательский ввод, изменять трансформацию объекта, прикладывать силы или импульсы к физическому телу, а также обращаться к контексту движка для выполнения действий уровня приложения. Например, в демонстрационной сцене скрипт может менять направление движения кинематической платформы или перезапускать сцену при нажатии клавиши.

2.3.6 Требования к базовой физической подсистеме

Физическая подсистема должна обеспечивать базовую обработку физических тел и столкновений. В системе должны поддерживаться динамические, статические и кинематические тела. Динамические тела должны реагировать на гравитацию, скорость, силы и импульсы. Статические тела должны использоваться как неподвижное окружение. Кинематические тела должны перемещаться заданным разработчиком способом и участвовать во взаимодействии с динамическими объектами.

Для проверки столкновений должен использоваться компонент коллайдера. В текущей версии системы основным типом коллайдера является коро-

бочный коллайдер. Он подходит для демонстрации падения кубов, взаимодействия с полом, стенами и движущейся платформой. Требование к физической подсистеме не включает моделирование сложных форм, мягких тел, суставов, транспортных средств и других возможностей полноценных физических движков.

Физическая система должна выполняться после пользовательских скриптов и до рендеринга. Такой порядок позволяет сначала обработать ввод и пользовательскую логику, затем обновить физическое состояние объектов, а после этого отобразить уже измененную сцену.

2.3.7 Требования к демонстрационной сцене

В составе программной системы должна быть подготовлена демонстрационная сцена, показывающая основные возможности движка. В качестве основного демонстрационного примера используется сцена `PhysicsScene`. Она должна показывать работу окна, камеры, света, рендера, физических тел, коллайдеров, пользовательских скриптов и отладочной визуализации.

Демонстрационная сцена должна содержать:

- свободную камеру с управлением клавиатурой и мышью;
- направленный источник света;
- статические объекты окружения: пол, стены и опору;
- динамические физические объекты, падающие под действием гравитации;
- кинематическую платформу, движущуюся по заданному сценарию;
- отладочную сетку и визуализацию коллайдеров;
- возможность перезапуска сцены по клавише.

2.4 Руководство по использованию программной системы

Данный подраздел описывает порядок работы с разрабатываемой программной системой с точки зрения Java-разработчика. В отличие от конечного пользовательского приложения, движок не предоставляет готовый визуальный интерфейс для создания сцен. Все действия выполняются в исходном

коде: разработчик создает приложение, описывает сцену, добавляет объекты и компоненты, после чего запускает программу.

2.4.1 Подготовка окружения

Для работы с программной системой необходимо установить JDK версии 25. В качестве внешней библиотеки используется LWJGL версии 3.3.6. В состав подключаемых модулей LWJGL должны входить:

- lwjgl – базовый модуль LWJGL;
- lwjgl-glfw – модуль для создания окна, OpenGL-контекста и обработки ввода;
- lwjgl-opengl – модуль для доступа к функциям OpenGL;
- lwjgl-stb – модуль для загрузки изображений и текстур.

Проект не использует Maven или Gradle. Поэтому разработчик может работать с ним двумя способами. Первый способ заключается в открытии проекта в IntelliJ IDEA и ручном добавлении JAR-файлов LWJGL в зависимости проекта. Второй способ заключается в ручной компиляции исходных файлов с помощью `javac` и запуске нужного класса через `java`.

При настройке проекта через IntelliJ IDEA необходимо выполнить следующие действия:

1. Открыть каталог проекта как обычный Java-проект.
2. Указать установленный JDK 25 в качестве SDK проекта.
3. Добавить JAR-файлы LWJGL 3.3.6 и JAR-файлы нативных библиотек для целевой операционной системы в раздел зависимостей проекта.
4. Убедиться, что рабочим каталогом запуска является корневой каталог проекта, в котором расположен каталог `res`.
5. Создать конфигурацию запуска для класса `common.showcase.PhysicsScene`.

Для ручной компиляции можно использовать следующий общий порядок команд. Команды предполагают, что все JAR-файлы LWJGL помещены в каталог `lib`, а компиляция выполняется из корня проекта.

Листинг 1 – Пример ручной компиляции проекта в Windows

```
1 dir /s /b src\*.java test\*.java > sources.txt
2 javac -encoding UTF-8 -d out -cp "lib/*" @sources.txt
3 java -cp "out;lib/*" common.showcase.PhysicsScene
```

Для Linux и macOS отличается только разделитель элементов classpath. Вместо точки с запятой используется двоеточие.

Листинг 2 – Пример ручной компиляции проекта в Linux и macOS

```
1 find src test -name "*.java" > sources.txt
2 javac -encoding UTF-8 -d out -cp "lib/*" @sources.txt
3 java -cp "out:lib/*" common.showcase.PhysicsScene
```

Если нативные библиотеки LWJGL не находятся автоматически через JAR-файлы, их необходимо указать через параметр `java.library.path`. Однако при использовании набора библиотек, полученного через конфигурацию LWJGL вместе с native JAR для соответствующей операционной системы, обычно достаточно добавить все JAR-файлы в classpath.

2.4.2 Создание приложения

Точкой входа в интерактивное приложение является обычный метод `main`. В нем создается конфигурация движка, экземпляр приложения, загружается стартовая сцена и запускается основной цикл. Пример минимальной структуры запуска приведен в листинге 2.3.

Листинг 3 – Создание приложения и запуск сцены

```
1 public static void main(String[] args) {
2     EngineConfig config = new EngineConfig()
3         .clearColor(0.05f, 0.07f, 0.08f, 1f)
4         .autoUpdateCameraAspect(true)
5         .debugOpenGLErrors(false);
6
7     Application app = new Application("interactive scene", config);
8     app.getWindow().setCursorDisabled();
9     app.context().sceneManager().load(createScene());
10    app.run();
11 }
```

В данном примере объект `EngineConfig` определяет параметры запуска. Метод `clearColor` задает цвет очистки окна перед отрисовкой кадра. Параметр `autoUpdateCameraAspect` включает автоматическое обновление соотноше-

ния сторон камеры при изменении размеров окна. Параметр `debugOpenGLErrors` позволяет отключить или включить дополнительные проверки ошибок OpenGL.

После создания приложения отключается отображение курсора, так как демонстрационная сцена использует свободное управление камерой мышью. Затем через менеджер сцен загружается сцена, созданная методом `createScene`, и вызывается метод `run`. После этого управление переходит основному циклу приложения.

2.4.3 Создание сцены и добавление систем

Сцена создается как экземпляр класса `Scene`. До добавления сущностей в нее необходимо подключить системы обработки. Порядок подключения систем важен, потому что каждая система выполняется в порядке добавления.

Листинг 4 – Создание сцены и подключение систем

```
1 private static Scene createScene() {  
2     Scene scene = new Scene();  
3  
4     scene.addSystem(new ScriptSystem());  
5  
6     scene.addSystem(new PhysicsSystem()  
7         .gravity(0f, -9.81f, 0f)  
8         .fixedTimeStep(1f / 60f)  
9         .maxSubSteps(5)  
10        .velocityIterations(12)  
11        .positionIterations(10)  
12        .debugDrawColliders(true));  
13  
14     scene.addSystem(new RenderSystem()  
15         .ambientColor(new Vec4(0.18f, 0.20f, 0.23f, 1f)));  
16  
17     scene.addSystem(new DebugRenderSystem());  
18  
19     return scene;  
20 }
```

Сначала добавляется `ScriptSystem`, поскольку пользовательские скрипты могут изменять состояние объектов, читать ввод, прикладывать силы или запрашивать смену сцены. После этого добавляется `PhysicsSystem`, которая обновляет физические тела и коллайдеры. Затем выполняется `RenderSystem`, по-

лучающая уже обновленные трансформации объектов. Последней добавляется `DebugRenderSystem`, отвечающая за вывод отладочной геометрии.

2.4.4 Добавление источника света

Для базового освещения сцены используется компонент `DirectionalLight`. Он задает направление света, цвет и интенсивность. Такой источник света не имеет положения в пространстве и имитирует удаленный источник, например солнце.

Листинг 5 – Создание направленного источника света

```
1 scene.addEntity(new Entity("sun")
2   .addComponent(new DirectionalLight()
3     .direction(-0.4f, -1f, -0.25f)
4     .color(new Vec4(1f, 0.94f, 0.82f, 1f))
5     .intensity(1.2f)));
```

Свет оформляется как обычная сущность с компонентом. Такой подход соответствует общей компонентной модели: даже служебные объекты сцены могут быть представлены через сущности и компоненты.

2.4.5 Создание камеры

Для отображения сцены необходима сущность с компонентом `Camera`. Чтобы камера имела положение и поворот, к сущности добавляется компонент `Transform`. Для управления камерой используется скрипт `FreecamController`.

Листинг 6 – Создание свободной камеры

```
1 scene.addEntity(new Entity("camera")
2   .addComponent(new Transform()
3     .position(0f, 4f, 9f)
4     .rotation(Quaternion.euler(-22f, 0f, 0f)))
5   .addComponent(new Camera()
6     .fov(60f)
7     .near(0.1f)
8     .far(80f)
9     .cullingMask(LAYER_WORLD | LAYER_PHYSICS)
10    .priority(10))
11   .addComponent(new FreecamController()
12     .speed(7f)
13     .sensitivity(18f)));
```

Компонент Camera задает угол обзора, ближнюю и дальнюю плоскости отсечения, маску видимости и приоритет. Приоритет нужен в случае, если в сцене присутствует несколько камер. Скрипт FreecamController считывает ввод пользователя и изменяет трансформацию камеры.

2.4.6 Создание статического объекта

Статический объект окружения можно создать без компонента Rigidbody. Для пола демонстрационной сцены достаточно задать трансформацию, коробочный коллайдер и компонент отображения меша.

Листинг 7 – Создание пола сцены

```
1 scene.addEntity(new Entity("floor")
2   .addComponent(new Transform()
3     .position(0f, -0.1f, 0f)
4     .scale(8f, 0.2f, 8f))
5   .addComponent(new BoxCollider()
6     .halfExtents(0.5f, 0.5f, 0.5f)
7     .friction(1f)
8     .restitution(0f))
9   .addComponent(new MeshRenderer()
10    .mesh(Meshes.cube())
11    .material(new Material()
12      .color(new Vec4(0.18f, 0.24f, 0.22f, 1f)))
13    .layer(LAYER_WORLD)
14    .boundsRadius(6f)));
```

В данном примере куб используется как масштабируемый параллелепипед. Компонент Transform растягивает его до размеров пола. Компонент BoxCollider задает область столкновения. Компонент MeshRenderer указывает геометрию, материал, слой отображения и радиус границ, используемый при обработке видимости.

2.4.7 Создание динамического физического тела

Динамический объект отличается от статического тем, что к нему добавляется компонент Rigidbody. Этот компонент хранит тип тела, массу, скорость, демпфирование, настройки гравитации и другие физические параметры.

Листинг 8 – Создание динамического куба

```

1 scene.addEntity(new Entity("falling-box")
2     .addComponent(new Transform()
3         .position(0f, 5f, 0f))
4     .addComponent(new Rigidbody()
5         .mass(1f)
6         .velocity(1f, 0f, 0f)
7         .linearDamping(0.05f)
8         .angularDamping(0.08f))
9     .addComponent(new BoxCollider()
10        .halfExtents(0.5f, 0.5f, 0.5f)
11        .friction(0.9f)
12        .restitution(0.1f))
13    .addComponent(new MeshRenderer()
14        .mesh(Meshes.cube())
15        .material(new Material()
16            .color(new Vec4(0.25f, 0.68f, 1f, 1f)))
17        .layer(LAYER_PHYSICS)
18        .boundsRadius(1f)));

```

После запуска сцены физическая система будет применять к такому объекту гравитацию, учитывать скорость, выполнять проверку столкновений с коллайдерами и изменять компонент Transform. Рендеринг будет использовать уже обновленное положение объекта.

2.4.8 Создание кинематического объекта

Кинематическое тело не падает под действием гравитации, но может перемещаться с заданной скоростью и взаимодействовать с динамическими телами. Такой объект удобно использовать для движущихся платформ и элементов окружения.

Листинг 9 – Создание кинематической платформы

```

1 scene.addEntity(new Entity("kinematic-platform")
2     .addComponent(new Transform()
3         .position(0f, 0.35f, 3.1f)
4         .scale(1.8f, 0.25f, 0.7f))
5     .addComponent(new Rigidbody()
6         .type(PhysicsBodyType.KINEMATIC)
7         .velocity(0.9f, 0f, 0f)
8         .useGravity(false))
9     .addComponent(new BoxCollider()
10        .halfExtents(0.5f, 0.5f, 0.5f)
11        .friction(0.9f))
12    .addComponent(new MeshRenderer()
13        .mesh(Meshes.cube())
14        .material(new Material()
15            .color(new Vec4(1f, 0.9f, 0.28f, 1f)))

```

```

16     .layer(LAYER_PHYSICS)
17     .boundsRadius(2f))
18     .addComponent(new PlatformBounceScript(-2.4f, 2.4f)));

```

В конце к сущности добавляется пользовательский скрипт PlatformBounceScript. Он следит за положением платформы и меняет направление скорости при достижении границ движения.

2.4.9 Создание пользовательского скрипта

Пользовательский скрипт создается наследованием от базового класса Script. В методе init обычно выполняется получение ссылок на компоненты сущности, а в методе loop описывается поведение, выполняемое каждый кадр.

Листинг 10 – Пример пользовательского скрипта движения платформы

```

1 private static final class PlatformBounceScript extends Script {
2     private final float minX;
3     private final float maxX;
4     private Rigidbody body;
5     private Transform transform;
6
7     private PlatformBounceScript(float minX, float maxX) {
8         this.minX = minX;
9         this.maxX = maxX;
10    }
11
12    @Override
13    public void init() {
14        body = entity().getComponent(Rigidbody.class);
15        transform = entity().getComponent(Transform.class);
16    }
17
18    @Override
19    public void loop(float dt) {
20        if (body == null || transform == null) return;
21
22        Vec3 position = transform.position();
23        Vec3 velocity = body.velocity();
24
25        if ((position.x <= minX && velocity.x < 0f) ||
26            (position.x >= maxX && velocity.x > 0f)) {
27            body.velocity(-velocity.x, velocity.y, velocity.z);
28        }
29    }
30 }

```

Данный пример показывает типовой способ расширения поведения объектов. Скрипт не изменяет устройство движка, не требует создания но-

вой системы и работает как компонент конкретной сущности. Благодаря этому прикладная логика остается отделенной от ядра программной системы.

2.4.10 Использование пользовательского ввода

Для чтения пользовательского ввода из скриптов используется класс Input. Например, в демонстрационной сцене клавиша R может использоваться для перезапуска сцены.

Листинг 11 – Пример обработки нажатия клавиши

```
1 private static final class PhysicsSceneControlScript extends Script {  
2     @Override  
3     public void loop(float dt) {  
4         if (!Input.GetKeyDown(KeyCode.R)) return;  
5  
6         EngineContext context = EngineContext.current();  
7         context.window().setCursorDisabled();  
8         context.sceneManager().requestReplace(createScene());  
9     }  
10 }
```

В этом примере используется однократное нажатие клавиши. При нажатии R получается текущий контекст движка, скрывается курсор и запрашивается замена активной сцены. Замена выполняется не немедленно, а через менеджер сцен, что предотвращает изменение структуры приложения в небезопасный момент цикла обновления.

2.4.11 Создание отладочной геометрии

Для наглядной проверки работы сцены может использоваться отладочный рендеринг. Например, можно каждый кадр выводить сетку на плоскости, чтобы лучше видеть движение физических объектов.

Листинг 12 – Пример вывода отладочной сетки

```
1 private static final class PhysicsGridScript extends Script {  
2     @Override  
3     public void loop(float dt) {  
4         for (int i = -4; i <= 4; i++) {  
5             DebugRenderer.line(  
6                 new Vec3(i, 0f, -4f),  
7                 new Vec3(i, 0f, 4f),  
8                 new Vec4(0.18f, 0.25f, 0.28f, 1f));  
9         }  
10     }  
11 }
```

```

10         DebugRenderer.line (
11             new Vec3(-4f, 0f, i),
12             new Vec3(4f, 0f, i),
13             new Vec4(0.18f, 0.25f, 0.28f, 1f));
14     }
15 }
16 }

```

Отладочная геометрия не является частью постоянной модели сцены. Она создается на кадр и используется для проверки положения объектов, границ коллайдеров, ориентации сцены и поведения физической подсистемы.

2.4.12 Общий порядок создания сцены

С учетом приведенных примеров общий порядок разработки интерактивной сцены выглядит следующим образом:

1. Создается конфигурация движка и экземпляр приложения.
2. Создается объект сцены.
3. В сцену добавляются системы обработки в требуемом порядке.
4. В сцену добавляется источник света.
5. В сцену добавляется камера и скрипт управления камерой.
6. Создаются статические объекты окружения: пол, стены и опоры.
7. Создаются динамические и кинематические физические объекты.
8. При необходимости добавляются пользовательские скрипты поведения.
9. Сцена загружается через менеджер сцен.
10. Запускается основной цикл приложения.

Такой порядок хорошо подходит для учебных и демонстрационных проектов, потому что вся сцена описывается в одном или нескольких Java-классах. Разработчик сразу видит, какие системы включены в сцену, какие сущности созданы, какие компоненты назначены объектам и какой код отвечает за поведение.

2.5 Требования к программному интерфейсу

Программный интерфейс системы должен быть доступен из Java-кода и не должен требовать от разработчика прямой работы с низкоуровневыми вызовами OpenGL для типовых операций создания сцены. Разработчик должен использовать готовые классы приложения, сцены, сущностей, компонентов, систем, материалов, мешей, текстур, ввода и времени.

Публичные классы должны поддерживать цепочечную настройку объектов, когда метод изменения параметра возвращает сам объект. Такой стиль используется при создании конфигурации, компонентов, материалов и систем. Он уменьшает объем вспомогательного кода и делает описание сцены более компактным.

Программный интерфейс должен позволять:

- создавать приложение с заданным заголовком окна и конфигурацией;
- загружать и заменять активную сцену;
- добавлять в сцену системы обработки;
- создавать сущности и назначать им компоненты;
- получать компоненты сущности по типу;
- создавать пользовательские скрипты поведения;
- получать состояние клавиатуры и мыши;
- загружать и использовать текстуры, материалы и шейдеры;
- создавать физические тела и коробочные коллайдеры;
- выводить отладочную геометрию при разработке сцены.

При этом интерфейс должен оставаться достаточно простым для учебного использования. Разработчик должен иметь возможность создать первую интерактивную сцену без предварительного изучения сложного визуального редактора, системы сборки, специализированного языка описания сцен или внешнего формата проекта.

2.6 Нефункциональные требования

2.6.1 Требования к программному обеспечению

Для разработки и запуска программной системы требуется JDK 25. В качестве внешней библиотеки используется LWJGL 3.3.6. Обязательными модулями LWJGL являются GLFW, OpenGL и STB. Система должна запускаться на настольных операционных системах Windows, Linux и macOS при наличии соответствующих нативных библиотек LWJGL и графического драйвера.

Так как система использует OpenGL и шейдеры GLSL версии 330 core, целевая рабочая станция должна поддерживать OpenGL 3.3 Core Profile или более новую совместимую версию. При отсутствии подходящего графического драйвера корректная работа графической подсистемы не гарантируется.

2.6.2 Требования к аппаратному обеспечению

Аппаратные требования программной системы определяются требованиями JDK 25, LWJGL, используемых нативных библиотек и графического драйвера. Для работы демонстрационной сцены необходим персональный компьютер или ноутбук с поддержкой OpenGL 3.3 Core Profile, клавиатурой и мышью. Отдельные численные требования к объему оперативной памяти, производительности процессора и видеосистемы в рамках технического задания не задаются, поскольку демонстрационная сцена не ориентирована на предельную нагрузку и используется для проверки архитектурных возможностей движка.

2.6.3 Требования к надежности

Программная система должна предсказуемо обрабатывать некорректное использование основных объектов. При передаче недопустимых значений в конфигурацию, компоненты или системы должны возникать диагностируемые исключения. Например, не допускается создание сущности с пу-

стым идентификатором, добавление пустого компонента, установка отрицательного фиксированного шага времени или установка неположительной массы физического тела.

Система должна корректно освобождать ресурсы при завершении приложения. После выхода из основного цикла должны быть уничтожены активные сцены, компоненты, системы, окно, графические ресурсы и контекст выполнения. Это требование особенно важно для приложений, использующих нативные ресурсы OpenGL и LWJGL.

Изменение состава сцены во время обновления должно выполняться безопасно. Если сущность, компонент или система добавляются или удаляются из пользовательского скрипта, операция должна быть отложена до момента, когда изменение структуры сцены не нарушит обход систем и сущностей.

2.6.4 Требования к сопровождаемости

Исходный код программной системы должен иметь модульную структуру. Классы ядра компонентной модели, компоненты, графическая подсистема, скрипты, физическая подсистема и демонстрационные сцены должны быть разделены по пакетам. Такое разделение необходимо для того, чтобы разработчик мог изучать и изменять отдельные части движка без просмотра всего проекта сразу.

Демонстрационные сцены должны использовать публичный программный интерфейс движка и выступать как практические примеры его применения. Это позволяет использовать их не только для проверки работоспособности, но и как основу пользовательской документации.

2.6.5 Требования к безопасности

Разрабатываемая программная система является локальной программной основой и не предусматривает сетевого взаимодействия, хранения учетных записей, обработки персональных данных и разграничения пользовательских ролей. Поэтому требования информационной безопасности огра-

ничиваются корректной обработкой внешних файлов ресурсов и предотвращением аварийного завершения при ошибках загрузки.

При работе с ресурсами должны быть диагностируемыми ошибки чтения файлов, компиляции шейдеров, загрузки текстур и создания графических объектов. Разработчик должен иметь возможность понять причину ошибки по сообщению исключения или отладочному выводу.

2.7 Требования к оформлению документации

Программная документация должна включать материалы, необходимые для понимания назначения, архитектуры, порядка сборки, запуска и использования программной системы. В составе пояснительной записки должны быть представлены анализ предметной области, техническое задание, технический проект и рабочий проект.

В техническом задании должны быть описаны назначение разработки, задачи, требования к программной системе, функциональные сценарии и порядок использования движка разработчиком. В техническом проекте должны быть раскрыты выбранные средства разработки, архитектура, структура пакетов, основные подсистемы и проектные решения. В рабочем проекте должны быть приведены спецификации основных классов, описание тестирования и результаты проверки работоспособности программной системы.

Текстовые материалы, рисунки, таблицы, листинги и графический материал должны оформляться в соответствии с требованиями, применяемыми к выпускным квалификационным работам по направлению подготовки 09.03.04 «Программная инженерия».

3 Технический проект

3.1 Общие сведения о программной системе

Разрабатываемая компонентно-ориентированная программная система предназначена для построения интерактивных приложений на языке Java. Система не является прикладной программой с фиксированным набором экранов и заранее заданной предметной областью. Ее основное назначение состоит в том, чтобы предоставить Java-разработчику программную основу для создания собственных сцен, объектов, компонентов, систем обработки и пользовательских сценариев поведения.

В рамках технического проекта программная система рассматривается как легковесный движок, ориентированный на программное создание интерактивных сцен. В составе разработки отсутствует визуальный редактор, поэтому основным интерфейсом взаимодействия с системой является Java API. Разработчик создает экземпляр приложения, задает конфигурацию, формирует сцену, регистрирует системы обработки и добавляет в сцену сущности с наборами компонентов. После запуска приложения основной цикл выполняет обновление активной сцены, обработку пользовательского ввода, выполнение скриптов, расчет базовой физики и отрисовку кадра.

Ключевая идея проектирования заключается в разделении объекта сцены и его функциональных признаков. Объект сцены представлен сущностью, а его состояние и поведение задаются компонентами. Например, объект камеры содержит компонент Transform, компонент Camera и скрипт свободного перемещения, а физический визуальный объект содержит Transform, Rigidbody, BoxCollider и MeshRenderer. Обработка сущностей выполняется специализированными системами: скриптовой, физической, графической и отладочной.

Такое устройство позволяет отказаться от глубокой иерархии классов для разных типов объектов. Вместо классов вида Player, Wall, Platform и LightObject используется сборка сущности из небольших независимых компонентов. В результате один и тот же компонент может переиспользоваться в

разных объектах сцены, а новая функциональность добавляется без изменения базового класса сущности.

Общая архитектура разрабатываемой программной системы представлена на рисунке 3.1.

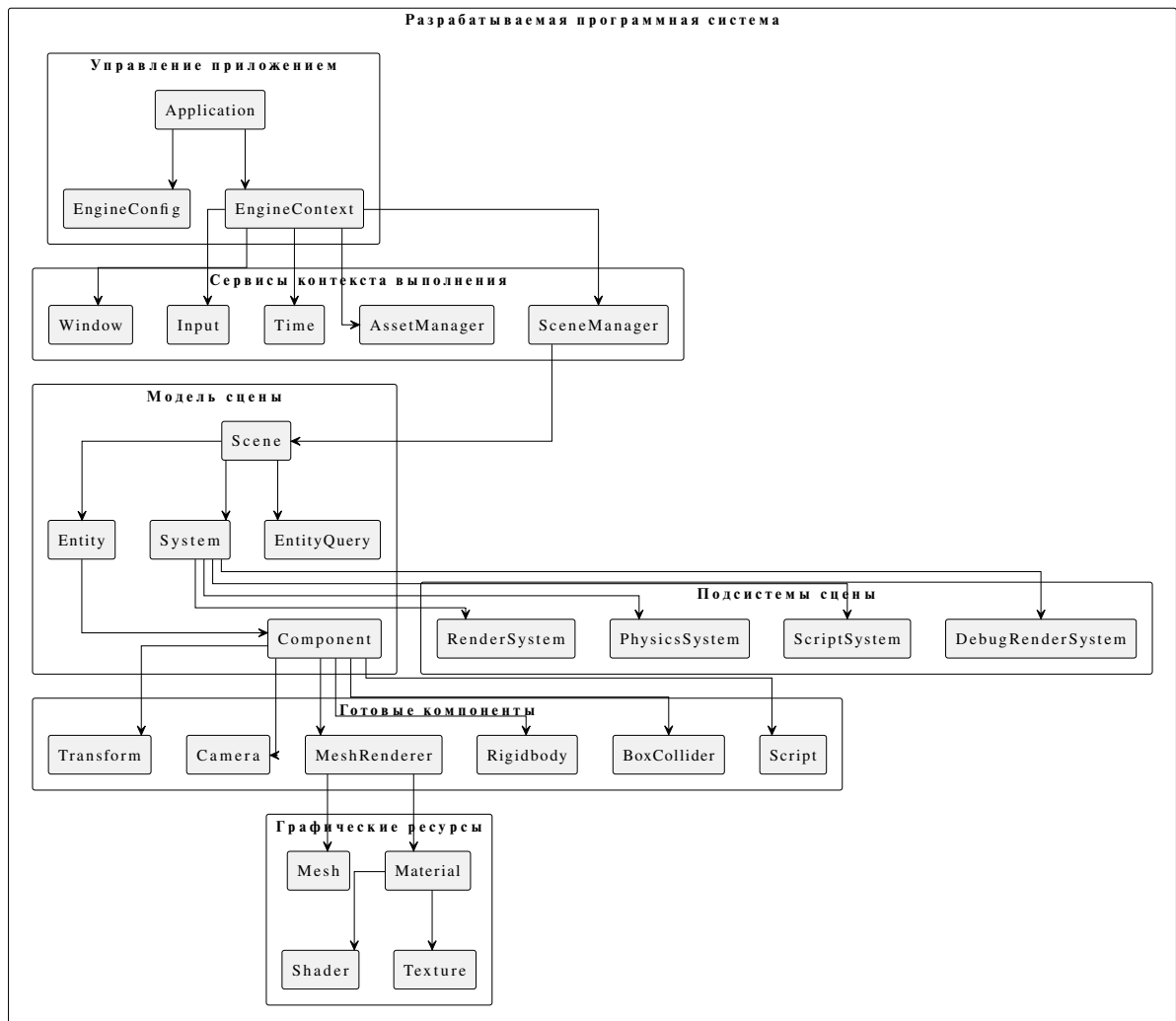


Рисунок 3.1 – Общая архитектура программной системы

В дальнейшем технический проект раскрывает не проект базы данных, так как база данных в системе не используется, а внутреннее устройство программной системы: структуру пакетов, принципы работы сцены, жизненный цикл приложения, устройство основных компонентов, графическую подсистему, подсистему ресурсов, скрипты, отладочную визуализацию и самостоятельно реализованную физическую подсистему.

3.2 Обоснование выбора технологий

Для реализации системы выбран набор технологий, который позволяет сохранить доступ к низкоуровневым возможностям графической подсистемы и при этом организовать архитектуру в привычной для Java-разработчика объектной форме. Основные технологии и их назначение приведены в таблице 3.1.

Таблица 3.1 – Используемые технологии и их назначение

Технология	Назначение	Причина использования
Java 25	основной язык реализации программной системы	обеспечивает объектно-ориентированную модель, строгую типизацию, кроссплатформенность на уровне JVM и удобство построения расширяемого API.
LWJGL 3.3.6	доступ из Java к нативным библиотекам	позволяет использовать OpenGL, GLFW и STB без перехода на C/C++, при этом не навязывает собственную архитектуру игрового движка.
OpenGL	графический API для вывода сцены	используется для создания GPU-ресурсов, передачи вершинных данных, настройки буферов и выполнения команд отрисовки.
GLSL	язык шейдерных программ	используется для описания вершинного и фрагментного шейдеров, расчета преобразований, текстурирования и базового освещения объектов.
GLFW	создание окна, OpenGL-контекста и обработка событий ввода	позволяет работать с настольным окном приложения, клавиатурой, мышью, событиями изменения размера окна и состоянием курсора.

Продолжение таблицы 3.1

Технология	Назначение	Причина использования
STB	загрузка изображений с диска	используется для загрузки текстур, применяемых материалами графической подсистемы.

3.2.1 Язык программирования Java

Java выбрана в качестве основного языка реализации, так как архитектура системы опирается на классы, наследование, инкапсуляцию и обобщенные типы. Компоненты сцены, системы обработки, менеджеры ресурсов и пользовательские скрипты представлены обычными Java-классами. Это делает программный интерфейс понятным для разработчика, знакомого с объектно-ориентированным программированием.

Дополнительным преимуществом является наличие виртуальной машины JVM. Приложение может запускаться на разных операционных системах при наличии установленного JDK, нативных библиотек LWJGL для целевой платформы и графического драйвера с поддержкой требуемой версии OpenGL. Для разрабатываемой системы это важно, поскольку она предназначена не для одной демонстрационной сцены, а для создания разных интерактивных приложений на общей программной основе.

3.2.2 Библиотека LWJGL

LWJGL используется как низкоуровневый слой доступа к нативным библиотекам. В рамках проекта задействованы GLFW, OpenGL и STB. При этом LWJGL не определяет структуру сцены, систему компонентов, жизненный цикл приложения, графическую очередь или физическую модель. Эти части реализуются в самой разрабатываемой системе.

Такое разделение удобно для проектирования. С одной стороны, система получает доступ к реальному графическому API и нативному окну. С другой стороны, прикладной разработчик не обязан напрямую работать с по-

вторяющимся низкоуровневым кодом: создание окна, управление ресурсами, обновление сцены, отрисовка и обработка ввода вынесены в отдельные классы движка.

3.2.3 OpenGL и GLSL

OpenGL применяется для вывода объектов сцены. На уровне графической подсистемы создаются вершинные массивы, буферы вершин, индексные буферы и выполняются команды отрисовки. Геометрия объекта хранится в классе Mesh, а ее GPU-представление создается и кэшируется при первом использовании в рендеринге.

Шейдеры написаны на языке GLSL. Вершинный шейдер получает данные вершины и матрицы модели, вида и проекции. Фрагментный шейдер использует параметры материала, текстуру, цвет и данные направленного источника света. Такой набор достаточен для демонстрации трехмерной сцены с материалами, текстурами, камерой и базовым направленным освещением.

3.2.4 GLFW и STB

GLFW используется для создания окна приложения, настройки OpenGL-контекста, обработки событий клавиатуры, мыши, прокрутки и изменения размеров окна. Состояние ввода сохраняется в объекте, доступном через фасад Input, поэтому пользовательские скрипты могут работать с клавишами и мышью без прямого обращения к callback-функциям GLFW.

STB применяется для загрузки изображений с диска. В текущей версии системы поддержана загрузка текстур, которые затем используются материалами. Загрузка трехмерных моделей из файлов не реализуется: геометрия задается программно, а для типовых случаев используются встроенные примитивы.

3.3 Внутреннее устройство программной системы

3.3.1 Структура пакетов проекта

Исходный код системы разделен на пакеты, каждый из которых отвечает за отдельную часть архитектуры. Такое разделение позволяет отделить ядро компонентной модели от графики, физики, управления приложением и пользовательских скриптов. Основные пакеты проекта приведены в таблице 3.2.

Таблица 3.2 – Основные пакеты проекта

Пакет	Назначение	Основные классы
common	математические и вспомогательные классы	Vec2, Vec3, Vec4, Mat4, Quaternion.
core	ядро компонентно-ориентированной модели	Entity, Component, System, EntityQuery, EntityObserver.
engine	управление приложением, сценами и ресурсами	Application, EngineContext, EngineConfig, Scene, SceneManager, AssetManager, Input, Time.
component	готовые компоненты объектов сцены	Transform, Camera, MeshRenderer, DirectionalLight, Script, Rigidbody, Collider, BoxCollider.
graphics	обертки над графическими ресурсами OpenGL	Window, Shader, Texture, MeshGpuResource.
system	системы обработки сцены	ScriptSystem, PhysicsSystem, RenderSystem, DebugRenderSystem.
script	переиспользуемые пользовательские скрипты	FreecamController.

Пакет core не зависит от OpenGL или LWJGL и содержит только общие понятия компонентной модели. Пакеты component и system строятся поверх ядра. Пакет engine объединяет инфраструктуру приложения, а пакет graphics содержит классы, непосредственно связанные с графическими ресурсами.

3.3.2 Основные элементы внутреннего устройства

Вместо проекта базы данных в данной работе рассматривается внутреннее устройство программной системы. Основные элементы системы приведены в таблице 3.3.

Таблица 3.3 – Основные элементы внутреннего устройства программной системы

Элемент	Роль в системе	Особенности реализации
Application	верхний управляющий объект приложения	создает окно, контекст выполнения, менеджеры ресурсов и сцен, запускает основной цикл.
EngineContext	контейнер общих сервисов	объединяет окно, ввод, время, менеджер ресурсов, менеджер сцен и конфигурацию.
Scene	центральный контейнер выполнения	хранит сущности, системы, запросы, индекс компонентов и очередь отложенных изменений.
Entity	объект сцены	представляет именованный контейнер компонентов, сам по себе не задает поведение объекта.
Component	признак или часть поведения сущности	имеет ссылку на сущность и методы жизненного цикла: присоединение, запуск, остановку и уничтожение.

Продолжение таблицы 3.3

Элемент	Роль в системе	Особенности реализации
System	обработчик сущностей сцены	выполняется в заданном порядке и получает доступ к сущностям через запросы.
EntityQuery	механизм выборки сущностей	позволяет системе работать только с сущностями, имеющими требуемый набор компонентов.
AssetManager	менеджер графических ресурсов	кэширует шейдеры и текстуры, освобождает созданные ресурсы при завершении работы.

Ключевыми элементами внутреннего устройства являются Scene, Entity, Component и System. Именно они определяют способ построения интерактивных приложений на базе разработанной системы. Диаграмма классов ядра компонентной модели представлена на рисунке 3.2.

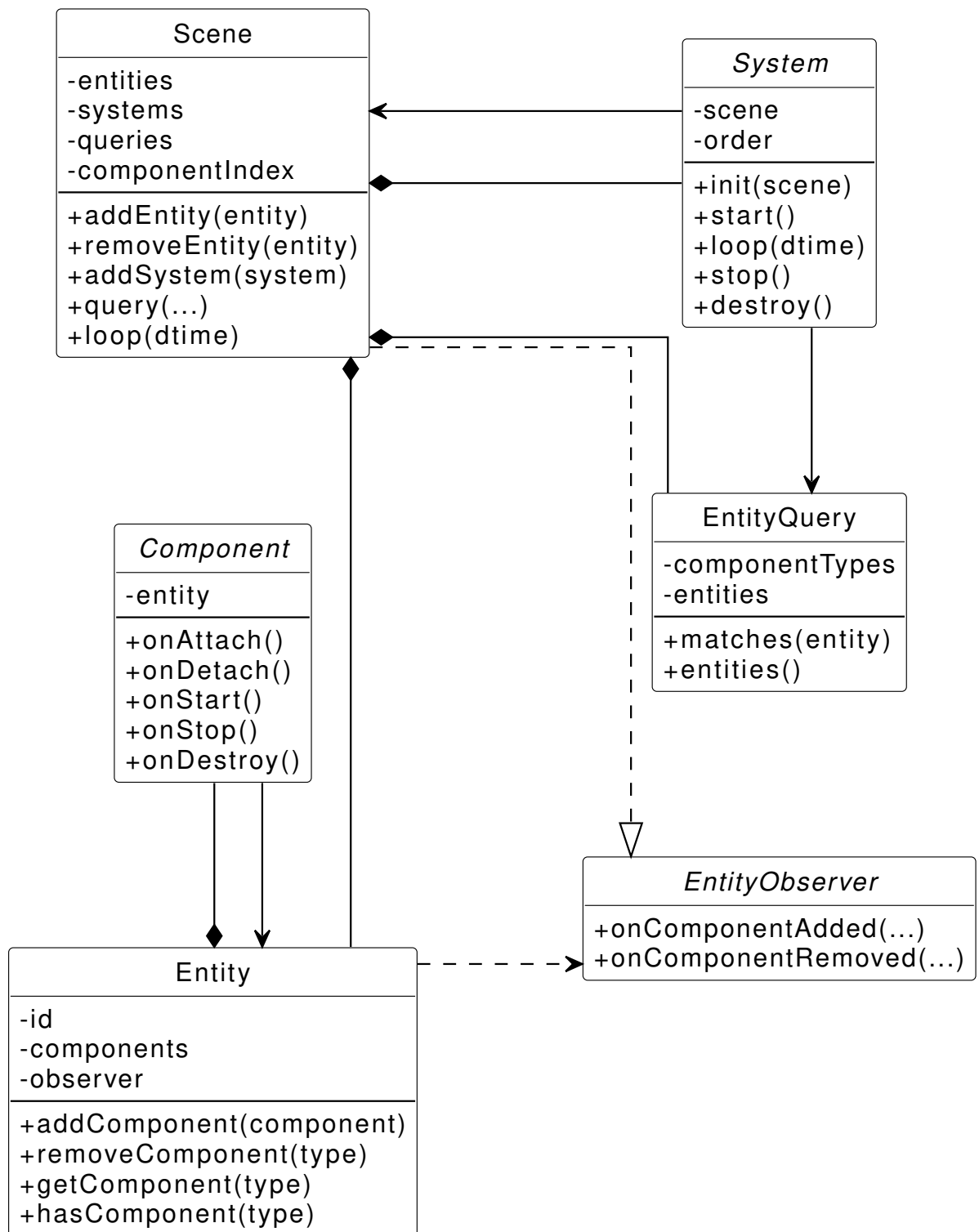


Рисунок 3.2 – Диаграмма классов ядра компонентной модели

3.3.3 Устройство приложения и контекста выполнения

Класс Application отвечает за создание и выполнение приложения. При создании экземпляра приложения формируется окно, создается объект EngineContext, инициализируются менеджер ресурсов, менеджер сцен, объект

времени и состояние ввода. После этого разработчик загружает сцену через SceneManager и вызывает метод запуска приложения.

Контекст выполнения используется для того, чтобы основные подсистемы могли обращаться к общим сервисам. Например, пользовательский скрипт может получить текущее состояние ввода через фасад Input, а графическая подсистема — обратиться к менеджеру ресурсов. Такое решение уменьшает количество ссылок, которые пришлось бы вручную передавать между скриптами, системами и сценами.

Конфигурация приложения вынесена в отдельный объект EngineConfig. В ней задаются параметры, влияющие на выполнение приложения: цвет очистки окна, фиксированный шаг времени, масштаб времени, автоматическое обновление соотношения сторон камер и режим проверки ошибок OpenGL. Благодаря этому настройки не смешиваются с логикой сцены.

3.3.4 Устройство сцены

Сцена является центральной частью выполнения интерактивного приложения. Внутри нее хранятся сущности, системы обработки, запросы сущностей, индекс компонентов и очередь отложенных структурных изменений. Такое устройство необходимо потому, что во время выполнения кадра состав сцены может меняться: скрипт может создать новый объект, удалить существующий, добавить компонент или запросить изменение активной сцены.

Сцена хранит сущности в отображении по строковому идентификатору. Это позволяет быстро получать объект по имени и не допускать добавления двух сущностей с одинаковым идентификатором. Системы обработки хранятся отдельно и сортируются по числовому порядку выполнения. Благодаря этому можно явно задавать, что система скриптов должна обновляться раньше физики, физика — раньше рендеринга, а отладочная отрисовка — после основной отрисовки сцены.

Для ускорения поиска объектов сцена поддерживает индекс компонентов. При добавлении сущности каждый ее компонент заносится в индекс. При удалении сущности или компонента индекс обновляется. Запрос EntityQuery

содержит набор требуемых компонентов и хранит множество сущностей, удовлетворяющих этому набору. Например, графическая система создает запрос по компонентам Transform и MeshRenderer, а физическая система — запросы по Transform, Rigidbody и наследникам Collider.

Отдельное значение имеет механизм отложенных структурных изменений. Если сцена в данный момент выполняет системы, немедленное добавление или удаление объекта может нарушить обход коллекций. Поэтому при обновлении сцены такие изменения помещаются в очередь. После завершения текущего прохода систем очередь применяется в безопасный момент. В результате пользовательские скрипты могут менять состав сцены, не создавая ошибок обхода и не нарушая согласованное состояние запросов.

При запуске сцена инициализирует сущности, строит индексы, перестраивает запросы и вызывает метод `init` у систем. Затем запускаются компоненты сущностей и системы обработки. При остановке сцены системы останавливаются, после чего останавливаются компоненты. При уничтожении сцены освобождаются системы, сущности, индексы и запросы.

3.3.5 Сущности, компоненты и системы обработки

Сущность представляет собой именованный контейнер компонентов. Она не содержит собственной прикладной логики и не наследуется от конкретных классов объектов. Смысл сущности определяется тем, какие компоненты к ней добавлены. Компонент Transform задает пространственное положение, Camera делает объект камерой, MeshRenderer позволяет отрисовать объект, Rigidbody включает объект в физическое моделирование, а наследник Script добавляет пользовательское поведение.

Базовый класс `Component` имеет жизненный цикл. При добавлении к сущности компонент получает ссылку на нее и может выполнить логику присоединения. При запуске сцены вызываются методы запуска, при остановке — методы остановки, при удалении — методы уничтожения. Это важно для компонентов, которые используют внутренние ресурсы или должны корректно реагировать на изменение состояния сцены.

Система обработки является объектом, который выполняется каждый кадр и работает с выбранными сущностями. В отличие от компонента, система обычно не принадлежит одной конкретной сущности. Она рассматривает сцену целиком и выбирает объекты по составу компонентов. Например, `RenderSystem` не хранится в каждом визуальном объекте, а обходит все сущности с компонентами `Transform` и `MeshRenderer`. Такой подход уменьшает связанность между объектами и централизует обработку однотипной логики.

3.3.6 Жизненный цикл приложения и сцены

Жизненный цикл одного кадра строится вокруг активной сцены. На каждом кадре приложение очищает окно, обновляет время, вызывает обновление активной сцены, применяет отложенные изменения, обновляет параметры камер, состояние ввода и окно. Последовательность выполнения одного кадра представлена на рисунке 3.3.

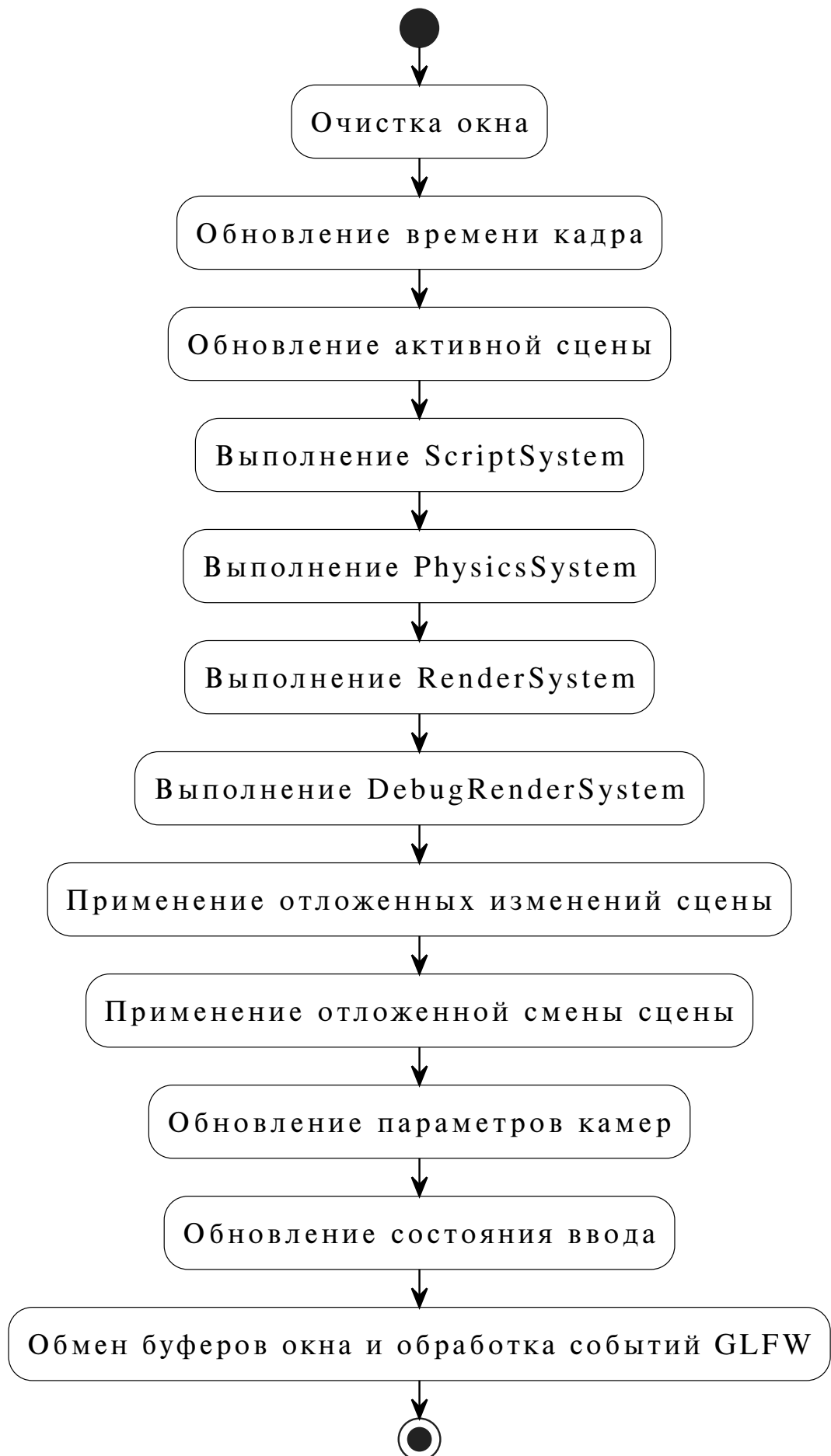


Рисунок 3.3 – Последовательность выполнения одного кадра приложения

Внутри обновления сцены выполняются зарегистрированные системы. Порядок выполнения задается числовым приоритетом системы. В демонстрационной сцене сначала выполняется ScriptSystem, затем PhysicsSystem, далее RenderSystem и DebugRenderSystem. Такой порядок выбран не случайно: скрипт может изменить управление объектом, физика обновляет положение тел, а рендеринг отображает уже обновленное состояние сцены.

3.4 Реализация основных подсистем

3.4.1 Подсистема управления сценами

Управление сценами выполняет SceneManager. Он хранит активную сцену и поддерживает операции загрузки, замены, отложенной загрузки и отложенной замены. При обычной загрузке старая сцена останавливается, а новая инициализируется и запускается. При замене старая сцена дополнительно уничтожается.

Отложенная замена нужна для случаев, когда команда смены сцены вызывается из системы или пользовательского скрипта во время кадра. Например, в демонстрационной сцене обработчик перезапуска может запросить создание новой сцены. Немедленное уничтожение текущей сцены в середине обновления было бы опасным, поэтому менеджер сцен применяет такое изменение после завершения текущего кадра.

3.4.2 Графическая подсистема

Графическая подсистема отвечает за отображение сцены средствами OpenGL. Ее центральным классом является RenderSystem. При инициализации система создает три запроса: к отрисовываемым объектам, к камерам и к направленным источникам света. Отрисовываемыми считаются сущности, имеющие компоненты Transform и MeshRenderer. Камерами считаются сущности с Transform и Camera. Направленное освещение задается компонентом DirectionalLight.

На каждом кадре `RenderSystem` сначала выбирает активную камеру. Если в сцене присутствует несколько камер, выбирается включенная камера с наибольшим приоритетом. Затем выбирается активный направленный источник света. После этого строится очередь команд отрисовки: система обходит все сущности с `Transform` и `MeshRenderer`, проверяет наличие меша и материала, активность рендерера, соответствие слоя объекта маске камеры и видимость объекта относительно текущей камеры.

Видимость объекта проверяется по ограничивающей сфере. Радиус этой сферы хранится в `MeshRenderer`. Объект переводится в пространство камеры, после чего отбрасывается, если находится за ближней или дальней плоскостью отсечения либо выходит за границы области видимости камеры с учетом радиуса. Для ортогографической и перспективной камеры используются разные расчеты границ, но общий смысл одинаков: объект, не попадающий в область обзора, не добавляется в очередь отрисовки.

После построения очередь сортируется. В системе используется сортировка по очереди рендеринга, идентификатору шейдера, идентификатору текстуры и объекту меша. Это уменьшает количество переключений графического состояния и делает отрисовку более предсказуемой. Данная оптимизация не является полноценным пакетированием графических команд, но она уже снижает число лишних смен шейдера, текстуры и вершинных данных.

Для каждой команды отрисовки система получает GPU-представление меша. Если меш используется впервые, создается `MeshGpuResource`, включающий VAO, VBO и EBO. Если ресурс уже создан и версия меша не изменилась, он переиспользуется. Если версия меша изменилась, старый GPU-ресурс уничтожается и создается новый. Благодаря этому геометрия не загружается в видеопамять на каждом кадре.

Перед вызовом OpenGL-команды отрисовки материал активирует шейдер и текстуру. Затем в шейдер передаются матрицы модели, вида и проекции, параметры направленного света, `ambient`-цвет и флаг наличия освещения. После этого привязывается VAO и вызывается `glDrawElements` с режимом `GL_TRIANGLES`. Таким образом, каждый визуальный объект проходит путь:

сущность сцены, компоненты Transform и MeshRenderer, команда рендеринга, GPU-ресурс меша, шейдер и команда OpenGL.

Основные классы графической подсистемы приведены в таблице 3.4.

Таблица 3.4 – Основные классы графической подсистемы

Класс	Назначение	Особенности реализации
RenderSystem	отрисовка объектов сцены	выбирает камеру и свет, строит и сортирует очередь рендеринга, вызывает OpenGL-команды.
MeshRenderer	визуальное представление сущности	связывает объект сцены с мешем, материалом, слоем, очередью рендеринга и радиусом видимости.
Camera	описание камеры	задает перспективную или ортографическую проекцию, приоритет, маску слоев, ближнюю и дальнюю плоскости отсечения.
Material	параметры отрисовки	содержит шейдер, цвет, текстуру и дополнительные uniform-параметры.
Mesh	геометрия объекта	хранит вершины, индексы и версию данных для контроля актуальности GPU-ресурса.
MeshGpuResource	GPU-представление меша	содержит идентификаторы VAO, VBO, EBO и количество индексов.
Shader	шейдерная программа	компилирует вершинный и фрагментный шейдеры и передает uniform-переменные.
Texture	текстура OpenGL	загружает изображение через STB или создает отладочную текстуру.

3.4.3 Подсистема ресурсов

Подсистема ресурсов представлена классом `AssetManager`. Он хранит загруженные шейдеры и текстуры в отображениях по строковому ключу. При повторном запросе ресурса с тем же ключом возвращается уже созданный объект. Это исключает повторную компиляцию одного и того же шейдера и повторную загрузку одной и той же текстуры.

Менеджер ресурсов также предоставляет базовый шейдер и отладочную текстуру. Базовый шейдер используется как стандартный вариант для материалов, которым не задана отдельная шейдерная программа. Отладочная текстура позволяет показать объект даже в тех случаях, когда разработчик не назначил собственную текстуру. При завершении работы приложения `AssetManager` освобождает все созданные шейдеры и текстуры.

3.4.4 Подсистема пользовательского ввода

Подсистема ввода получает события от `GLFW` и сохраняет их в состоянии ввода. Поддерживаются клавиатура, кнопки мыши, положение курсора, смещение мыши и прокрутка. Прикладной код обращается к этим данным через фасад `Input`, поэтому пользовательские скрипты не зависят от конкретной реализации `callback`-функций окна.

Примером использования подсистемы ввода является `FreecamController`. Он перемещает камеру по клавишам `W`, `A`, `S`, `D`, `SPACE` и `LEFT_SHIFT`, а поворот камеры определяет по смещению мыши. Такой скрипт одновременно проверяет работу ввода, трансформации и камеры.

3.4.5 Подсистема пользовательских скриптов

Пользовательские сценарии поведения реализуются через наследование от класса `Script`. Так как `Script` является компонентом, он добавляется к сущности так же, как `Camera`, `MeshRenderer` или `Rigidbody`. Система `ScriptSystem` находит компоненты-наследники `Script` и вызывает их методы жизненного цикла: `init`, `start`, `loop`, `stop`, `destroy`.

Скрипт может получать доступ к другим компонентам своей сущности и изменять их состояние. Например, скрипт движущейся платформы получает Rigidbody и задает ему скорость, а скрипт управления сценой отслеживает нажатие клавиши и запрашивает перезапуск демонстрационной сцены через SceneManager.

3.4.6 Подсистема отладочной визуализации

Подсистема отладочной визуализации предназначена для вывода вспомогательной геометрии поверх основной сцены. Она включает DebugRenderer и DebugRenderSystem. Разработчик или другая подсистема может добавить линии в очередь отладочной отрисовки, после чего DebugRenderSystem выводит их средствами OpenGL.

В демонстрационной физической сцене отладочная визуализация используется для сетки и границ коллайдеров. Это помогает проверять соответствие визуальной геометрии и физической формы объекта. Подсистема не заменяет основную графику, а служит инструментом разработки и проверки поведения сцены.

3.5 Реализация физической подсистемы

3.5.1 Общая структура физической подсистемы

Физическая подсистема реализована самостоятельно, без использования готового физического движка. Она предназначена для базовой демонстрации движения твердых тел и обработки столкновений в интерактивной сцене. В текущей версии системы поддерживаются динамические, статические и кинематические тела, гравитация, силы, моменты сил, линейная и угловая скорость, демпфирование, упругость, трение, слои столкновений, маски столкновений и коллайдер типа BoxCollider.

Название BoxCollider в тексте работы следует понимать как коллайдер формы параллелепипеда, то есть ориентированный ограничивающий параллелепипед. Его положение, ориентация и масштаб вычисляются на основе

матрицы модели компонента Transform. В отличие от осево-ориентированной границы, такой коллайдер может поворачиваться вместе с объектом сцены.

Основные классы физической подсистемы приведены в таблице 3.5.

Таблица 3.5 – Основные классы физической подсистемы

Класс	Назначение	Особенности реализации
PhysicsSystem	расчет физики и столкновений	использует фиксированный шаг, аккумулятор времени, итерации решения скоростей и позиций.
Rigidbody	физическое тело	хранит массу, обратную массу, скорость, угловую скорость, силу, момент силы, тип тела и параметры демпфирования.
Collider	базовый класс области столкновения	задает смещение, слой, маску столкновений, трение, упругость, флаг триггера и активность.
BoxCollider	коллайдер формы параллелепипеда	хранит половинные размеры по трем осям и используется для построения коллайдера формы параллелепипеда.
PhysicsBodyType	тип физического тела	разделяет тела на динамические, статические и кинематические.

Физическая система при инициализации создает два запроса: один для сущностей с Transform и Rigidbody, второй для сущностей с Transform и наследниками Collider. На каждом кадре она накапливает прошедшее время и выполняет один или несколько физических подшагов фиксированной длины. Число подшагов ограничивается параметром maxSubSteps, чтобы длинный кадр не приводил к бесконечной попытке “догнать” время моделирования.

3.5.2 Математическая модель движения

Поступательное движение динамического тела описывается классическими дифференциальными уравнениями движения материальной точки:

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}, \quad \frac{d\mathbf{v}}{dt} = \mathbf{a}, \quad \mathbf{a} = \frac{\mathbf{F}}{m} + s_g \mathbf{g},$$

где $\mathbf{x}$ – положение тела, $\mathbf{v}$ – линейная скорость, $\mathbf{a}$ – ускорение, $\mathbf{F}$ – сумма внешних сил, m – масса тела, $\mathbf{g}$ – вектор гравитации, s_g – коэффициент влияния гравитации для конкретного тела.

Для вращательного движения используется угловая скорость $\boldsymbol{\omega}$ и момент силы $\boldsymbol{\tau}$:

$$\frac{d\boldsymbol{\omega}}{dt} = \mathbf{I}^{-1} \boldsymbol{\tau},$$

где $\mathbf{I}$ – тензор инерции тела. Для тела с параллелепипедным коллайдером в коде используется приближение инерции прямоугольного параллелепипеда. Если ширина, высота и глубина равны w , h и d , то главные моменты инерции вычисляются как:

$$I_x = \frac{m}{12}(h^2 + d^2), \quad I_y = \frac{m}{12}(w^2 + d^2), \quad I_z = \frac{m}{12}(w^2 + h^2).$$

В реализации хранится не сам тензор инерции, а применяется обратная инерция по локальным осям параллелепипеда. Это нужно при расчете углового изменения скорости от импульса, приложенного не в центре масс.

3.5.3 Метод численного интегрирования

Физика обновляется с фиксированным шагом Δt . В демонстрационной сцене используется шаг 1/60 секунды. Внутри одного физического шага сначала обновляются скорости по силам, затем по обновленным скоростям изменяются положение и поворот объекта. Такой порядок соответствует полуживному методу Эйлера:

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t, \quad \mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_{n+1} \Delta t.$$

Для угловой скорости используется аналогичная схема:

$$\boldsymbol{\omega}_{n+1} = \boldsymbol{\omega}_n + \mathbf{I}^{-1} \boldsymbol{\tau}_n \Delta t.$$

После этого из угловой скорости вычисляется малый поворот за текущий шаг:

$$\Delta\varphi = |\boldsymbol{\omega}_{n+1}| \Delta t.$$

Ось поворота определяется нормированным вектором угловой скорости, после чего создается кватернион дополнительного поворота. Новый поворот объекта получается умножением текущего кватерниона на кватернион приращения с последующей нормализацией. Такой способ удобен для трехмерной сцены, так как позволяет избежать проблем с накоплением углов Эйлера.

Перед применением скорости используются линейное и угловое демпфирование:

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+1} \max(0, 1 - k_l \Delta t), \quad \boldsymbol{\omega}_{n+1} = \boldsymbol{\omega}_{n+1} \max(0, 1 - k_a \Delta t),$$

где k_l – коэффициент линейного демпфирования, а k_a – коэффициент углового демпфирования. Также в системе ограничиваются максимальная линейная и максимальная угловая скорости. Это уменьшает вероятность неустойчивого поведения при больших силах или ошибочно заданных параметрах сцены.

3.5.4 Обнаружение столкновений

Обнаружение столкновений состоит из двух этапов. На первом этапе выполняется быстрая проверка пересечения осево-ориентированных ограничивающих объемов (AABB). Если эти объемы не пересекаются, более точная

проверка не выполняется. Такой этап нужен для раннего отбрасывания пар объектов, которые находятся далеко друг от друга.

На втором этапе для двух коллайдеров формы параллелепипеда используется проверка по теореме разделяющих осей. Каждый `VoxCollider` преобразуется в коллайдер формы параллелепипеда: вычисляются центр, три локальные оси в мировых координатах и половинные размеры с учетом масштаба объекта. Далее проверяются оси первого параллелепипеда, оси второго параллелепипеда и попарные векторные произведения этих осей. Всего для пары трехмерных параллелепипедов рассматривается до пятнадцати осей.

Для каждой оси вычисляются радиусы проекции двух параллелепипедов и расстояние между их центрами вдоль этой оси. Если сумма радиусов меньше расстояния между центрами, между объектами существует разделяющая ось, и столкновения нет. Если пересечение найдено по всем осям, пара считается сталкивающейся. Ось с минимальным перекрытием используется как нормаль столкновения, а величина перекрытия – как глубина проникновения.

После определения нормали и глубины проникновения формируется набор контактных точек. Для этого вершины, близкие к контактной плоскости, проецируются на эту плоскость и проверяются на принадлежность второму параллелепипеду. Если точек получается слишком много, они сокращаются до ограниченного количества, чтобы решатель столкновений не выполнял лишнюю работу.

3.5.5 Разрешение столкновений

Разрешение столкновений выполняется итерационно. Сначала несколько раз обрабатываются скорости, затем несколько раз выполняется позиционная коррекция. Такой подход не дает идеального аналитического решения для всех контактов сразу, но хорошо подходит для интерактивной демонстрационной сцены.

Для контактной точки вычисляется относительная скорость:

$$\mathbf{v}_{rel} = (\mathbf{v}_b + \boldsymbol{\omega}_b \times \mathbf{r}_b) - (\mathbf{v}_a + \boldsymbol{\omega}_a \times \mathbf{r}_a),$$

где $\mathbf{r}_a$ и $\mathbf{r}_b$ – радиус-векторы от центра масс тел до контактной точки. Проекция этой скорости на нормаль контакта показывает, сближаются ли тела вдоль нормали.

Нормальный импульс вычисляется по формуле:

$$j_n = \frac{b - \mathbf{v}_{rel} \cdot \mathbf{n}}{m_a^{-1} + m_b^{-1} + D_a + D_b},$$

где $\mathbf{n}$ – нормаль контакта, b – скоростная поправка, учитывающая упругость и баумгартовскую коррекцию проникновения, D_a и D_b – угловые добавки, возникающие из-за приложения импульса не в центре масс. Если знаменатель равен нулю или тела не должны двигаться, импульс не применяется.

После нормального импульса рассчитывается касательное направление и импульс трения. Его величина ограничивается максимальным значением, пропорциональным нормальному импульсу и коэффициенту трения. Благодаря этому объект может не только отскакивать от поверхности, но и постепенно терять касательную скорость при контакте.

Позиционная коррекция устраняет остаточное взаимное проникновение тел. В системе используется небольшая допустимая глубина s , после которой часть проникновения распределяется между телами пропорционально обратным массам:

$$C = \min(0,05, \max(0, p - s) \cdot 0,2),$$

где p – глубина проникновения. Если одно тело статическое, вся коррекция фактически приходится на динамическое тело. Такой способ не делает физику абсолютно точной, но предотвращает заметное накопление проникновений между объектами.

3.5.6 Границы текущего этапа разработки

Физическая подсистема предназначена для базовой обработки твердых тел и столкновений в демонстрационных интерактивных сценах. На текущем этапе поддерживается параллелепипедный тип коллайдера, динамические, статические и кинематические тела, импульсное разрешение столкновений, трение, упругость, ограничение скорости и режим сна для устойчиво покоящихся тел.

При этом система не ставит целью заменить промышленный физический движок. В текущую область разработки не входят составные соединения тел, шарнирные ограничители, непрерывное обнаружение столкновений для очень быстрых объектов, коллайдеры произвольной сетки и сложные материалы. Такое ограничение соответствует задачам ВКР: необходимо показать собственную архитектуру физической подсистемы и ее совместную работу со сценой, компонентами, скриптами и рендерингом.

3.6 Демонстрационная сцена PhysicsScene

Для проверки проектных решений используется демонстрационная сцена PhysicsScene. Она выбрана в качестве основной, так как одновременно задействует сцены, сущности и компоненты, рендеринг, пользовательский ввод, скрипты, базовую физику и отладочную визуализацию.

Структура демонстрационной сцены показана на рисунке 3.4.

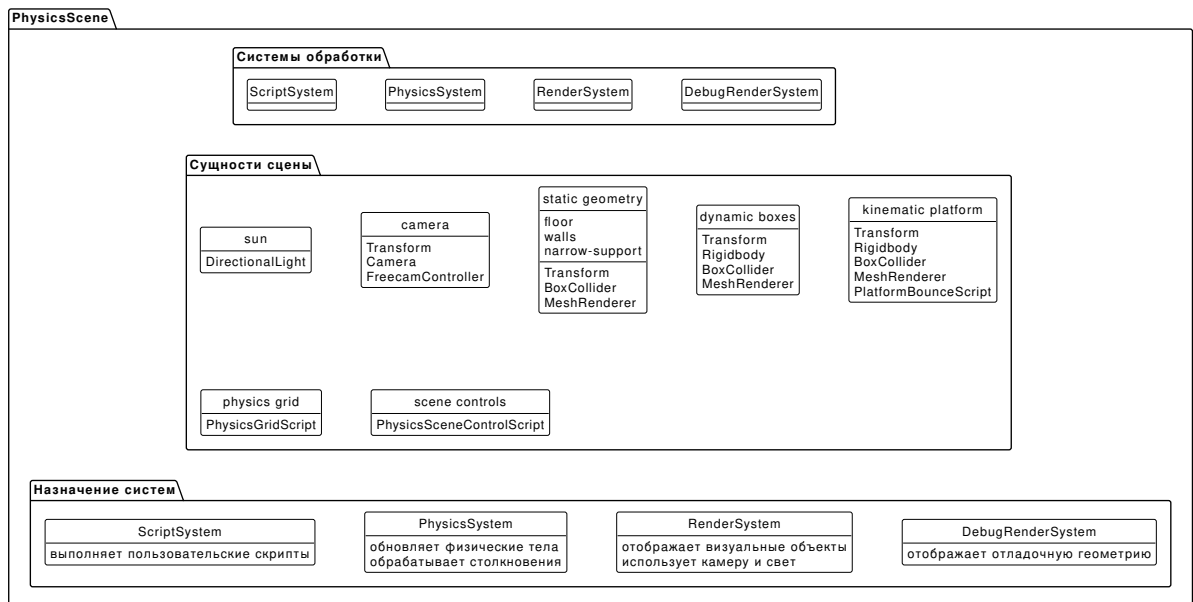


Рисунок 3.4 – Структура демонстрационной сцены

В сцене сначала регистрируются системы обработки. Их порядок выбран так, чтобы пользовательские скрипты могли изменить параметры объектов до физического шага, физическая система успела обновить трансформации тел до отрисовки, а отладочная визуализация выполнялась после основной графики. Упрощенный пример регистрации систем приведен ниже.

```

1 scene.addSystem(new ScriptSystem());
2
3 scene.addSystem(new PhysicsSystem()
4     .gravity(0f, -9.81f, 0f)
5     .fixedTimeStep(1f / 60f)
6     .maxSubSteps(5)
7     .velocityIterations(12)
8     .positionIterations(10)
9     .debugDrawColliders(true));
10
11 scene.addSystem(new RenderSystem()
12     .ambientColor(new Vec4(0.18f, 0.20f, 0.23f, 1f)));
13
14 scene.addSystem(new DebugRenderSystem());
  
```

После регистрации систем сцена наполняется сущностями. Источник света представлен сущностью **sun** с компонентом **DirectionalLight**. Камера представлена сущностью **camera**, содержащей **Transform**, **Camera** и **FreecamController**. Пол, стены и опора представлены статическими объектами с компонентами **Transform**, **BoxCollider** и **MeshRenderer**. Динамические тела содержат **Rigidbody**, поэтому падают под действием гравитации и сталкиваются с окружением. Ки-

нематическая платформа имеет тип KINEMATIC, движется заданной скоростью и меняет направление с помощью пользовательского скрипта.

Основные объекты демонстрационной сцены приведены в таблице 3.6.

Таблица 3.6 – Основные объекты демонстрационной сцены

Объект сцены	Компоненты	Назначение
sun	DirectionalLight	создает направленное освещение сцены.
camera	Transform, Camera, FreecamController	обеспечивает обзор сцены и перемещение пользователя с клавиатуры и мыши.
floor, walls, narrow-support	Transform, BoxCollider, MeshRenderer	формируют статическое окружение, с которым взаимодействуют динамические тела.
dynamic boxes	Transform, Rigidbody, BoxCollider, MeshRenderer	демонстрируют падение, столкновения, трение, вращение и различие параметров физических тел.
kinematic-platform	Transform, Rigidbody, BoxCollider, MeshRenderer, PlatformBounceScript	демонстрирует кинематическое тело, движущееся заданной скоростью и взаимодействующее с динамическими телами.
physics-grid	PhysicsGridScript	рисует отладочную сетку на плоскости сцены.
scene-controls	PhysicsSceneControlScript	обрабатывает перезапуск сцены по клавише.

Демонстрационная сцена проверяет несколько проектных решений. Во-первых, она показывает, что объект можно собрать из независимых ком-

понентов. Во-вторых, она подтверждает правильность порядка выполнения систем: скрипт платформы изменяет параметры движения, физическая система учитывает их при расчете столкновений, а рендеринг получает уже обновленные трансформации. В-третьих, сцена проверяет отложенную замену сцены: пользовательский скрипт может запросить перезапуск, не разрушая активную сцену в середине кадра.

3.7 Выводы по разделу

В рамках технического проекта определено внутреннее устройство разрабатываемой компонентно-ориентированной программной системы. Обоснован выбор Java, LWJGL, OpenGL, GLSL, GLFW и STB как технологической основы проекта. Вместо проекта базы данных описаны реальные элементы устройства движка: приложение, контекст выполнения, сцена, сущности, компоненты, системы обработки, запросы сущностей, ресурсы, графические классы и физические компоненты.

Подробно рассмотрено устройство сцены, включая хранение сущностей и систем, индексацию компонентов, запросы сущностей, жизненный цикл и механизм отложенных структурных изменений. Описана графическая подсистема: выбор активной камеры, построение очереди рендеринга, проверка видимости, сортировка команд, кэширование GPU-ресурсов мешей и передача данных в шейдеры. Отдельно рассмотрена физическая подсистема, реализованная без готового физического движка: приведены уравнения движения, метод полуявного интегрирования Эйлера, обнаружение столкновений коллайдеров формы параллелепипеда и импульсное разрешение контактов.

В качестве практического подтверждения проектных решений рассмотрена сцена PhysicsScene. Она демонстрирует совместную работу камеры, освещения, рендеринга, ввода, пользовательских скриптов, физических тел, столкновений и отладочной визуализации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Грегори, Д. Архитектура игровых движков / Д. Грегори. – 3-е изд. – Boca Raton : CRC Press, 2018. – 1240 с. – ISBN 978-1-138-03545-4. – Текст : непосредственный.
2. Нистром, Р. Шаблоны игрового программирования : сайт. – URL: <https://gameprogrammingpatterns.com/> (дата обращения: 17.05.2026).
3. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – Санкт-Петербург : Питер, 2020. – 366 с. – ISBN 978-5-4461-1595-0. – Текст : непосредственный.
4. Эриксон, К. Обнаружение столкновений в реальном времени / К. Эриксон. – San Francisco : Morgan Kaufmann, 2005. – 632 с. – ISBN 978-1-55860-732-3. – Текст : непосредственный.
5. Миллингтон, И. Разработка физического движка для игр / И. Миллингтон. – 2-е изд. – Boca Raton : CRC Press, 2010. – 542 с. – ISBN 978-0-12-381976-5. – Текст : непосредственный.
6. Eberly, D. H. 3D Game Engine Design : A Practical Approach to Real-Time Computer Graphics / D. H. Eberly. – 2nd ed. – Boca Raton : CRC Press, 2006. – 1040 p. – ISBN 978-0-12-229063-3. – Текст : непосредственный.
7. Akenine-Moller, T. Real-Time Rendering / T. Akenine-Moller, E. Haines, N. Hoffman. – 4th ed. – Boca Raton : CRC Press, 2018. – 1198 p. – ISBN 978-1-138-62700-0. – Текст : непосредственный.
8. Java Platform, Standard Edition and Java Development Kit. Version 25 API Specification : сайт. – URL: <https://docs.oracle.com/en/java/javase/25/docs/api/index.html> (дата обращения: 17.05.2026).
9. The Java Language Specification. Java SE 25 Edition : сайт. – URL: <https://docs.oracle.com/javase/specs/jls/se25/html/index.html> (дата обращения: 17.05.2026).

10. The Java Virtual Machine Specification. Java SE 25 Edition : сайт. – URL: <https://docs.oracle.com/javase/specs/jvms/se25/html/index.html> (дата обращения: 17.05.2026).

11. LWJGL – Lightweight Java Game Library : сайт. – URL: <https://www.lwjgl.org/> (дата обращения: 17.05.2026).

12. LWJGL 3.3.6 Release : сайт. – URL: <https://www.lwjgl.org/browse/release/3.3.6> (дата обращения: 17.05.2026).

13. LWJGL 3 API Documentation : сайт. – URL: <https://javadoc.lwjgl.org/> (дата обращения: 17.05.2026).

14. GLFW Documentation : сайт. – URL: <https://www.glfw.org/docs/> (дата обращения: 17.05.2026).

15. GLFW. Window guide : сайт. – URL: https://www.glfw.org/docs/3.4/window_guide.html (дата обращения: 17.05.2026).

16. GLFW. Input guide : сайт. – URL: https://www.glfw.org/docs/3.4/input_guide.html (дата обращения: 17.05.2026).

17. OpenGL – The Industry’s Foundation for High Performance Graphics : сайт. – URL: <https://www.khronos.org/opengl/> (дата обращения: 17.05.2026).

18. OpenGL 3.3 Reference Pages : сайт. – URL: <https://opengl.org/sdk/docs/man3/> (дата обращения: 17.05.2026).

19. OpenGL Wiki. Rendering Pipeline Overview : сайт. – URL: https://wikis.khronos.org/opengl/Rendering_Pipeline_Overview (дата обращения: 17.05.2026).

20. Kessenich, J. The OpenGL Shading Language. Language Version 3.30 / J. Kessenich, D. Baldwin, R. Rost : сайт. – URL: <https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.3.30.pdf> (дата обращения: 17.05.2026).

21. stb single-file public domain libraries for C/C++ : сайт. – URL: <https://github.com/nothings/stb> (дата обращения: 17.05.2026).

22. stb_image.h : сайт. – URL: https://github.com/nothings/stb/blob/master/stb_image.h (дата обращения: 17.05.2026).
23. libGDX – Cross-platform Java game development framework : сайт. – URL: <https://libgdx.com/> (дата обращения: 17.05.2026).
24. jMonkeyEngine : сайт. – URL: <https://jmonkeyengine.org/> (дата обращения: 17.05.2026).
25. Processing : сайт. – URL: <https://processing.org/> (дата обращения: 17.05.2026).
26. Unity Manual. Add components to GameObjects : сайт. – URL: <https://docs.unity3d.com/6000.0/Documentation/Manual/unity-components.html> (дата обращения: 17.05.2026).
27. ECS for Unity : сайт. – URL: <https://unity.com/ecs> (дата обращения: 17.05.2026).
28. PlantUML Documentation : сайт. – URL: <https://plantuml.com/> (дата обращения: 17.05.2026).
29. PlantUML. Class Diagram : сайт. – URL: <https://plantuml.com/class-diagram> (дата обращения: 17.05.2026).
30. Unified Modeling Language 2.5.1 Specification : сайт. – URL: <https://www.omg.org/spec/UML/2.5.1/> (дата обращения: 17.05.2026).